ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

MÔN TRÍ TUỆ NHÂN TẠO NÂNG CAO

ĐỒ ÁN CUỐI KỲ
NGHIÊN CỨU KIẾN TRÚC VÀ SỰ PHÁT TRIỂN DÒNG MÔ HÌNH QWEN

Nhóm thực hiện:

25C15017 – Lý Kim Long

25C15031 – Nguyễn Hoàng Anh

25C15032 – Nguyễn Quốc Anh

25C15047 – Chu Hải Linh

25C15052 – Nguyễn Đặng Nhật Minh

Giảng viên giảng dạy:

Thầy Bùi Duy Đăng

Thầy Lê Trung Nghĩa

Thành phố Hồ Chí Minh, tháng 03 năm 2026

Mục lục

I. Giới thiệu thành viên và phân chia công việc.....5
1. Giới thiệu thành viên.....5
2. Phân chia công việc.....5
II. Giới thiệu tổng quan về đồ án..... 6
1. Bối cảnh nghiên cứu.....6
2. Mục tiêu nghiên cứu.....6
III. Kiến trúc và sự phát triển của Qwen..... 7
1. Qwen1 – Kiến trúc nền tảng.....7
1. Tổng quan.....7
2. Tokenization và biểu diễn đầu vào.....7
3. Kiến trúc tổng thể của Transformer trong Qwen..... 9
4. Cơ chế Self-Attention.....12
5. Positional Encoding.....14
6. Normalization.....16
7. Feed Forward Network.....19
8. Embedding và Output Layer..... 21
9. Cơ chế mở rộng context trong kiến trúc.....23
2. Qwen2 Bước tiến quan trọng..... 24
1. Tổng quan.....24
2. Bộ mã hóa token & Mô hình.....24
a) Bộ mã hóa token..... 24
b) Kiến trúc mô hình..... 24
Nguyên lý hoạt động và Công thức.....25
Các cải tiến cốt lõi trong Qwen2 MoE.....26
Cấu hình chi tiết (Qwen2-57B-A14B).....27
2.2.3 Cấu hình mô hình.....28
2.3. Tiền huấn luyện.....29
2.3.1 Dữ liệu tiền huấn luyện.....29
2.3.2 Huấn luyện ngữ cảnh dài.....30
2.4. Hậu huấn luyện.....30

2.4.1 Dữ liệu hậu huấn luyện..... 30
2.4.2 Tinh chỉnh có giám sát (SFT).....31
2.4.3 Học tăng cường từ phản hồi con người (RLHF).....31
2.5. Đánh giá..... 31
2.5.1 Mô hình ngôn ngữ nền tảng..... 31
2.5.2 Mô hình được tinh chỉnh theo hướng dẫn.....31
2.6 Kết luận..... 32
3. Qwen2.5 - Cải thiện đáng kể.....32
1. Tổng quan.....32
2. Những cải tiến chính..... 33
3. Kiến trúc và bộ mã hóa (architecture & tokenizer).....33
4. Tiền huấn luyện (pre-training)..... 34
5. Sau huấn luyện (post-training)..... 35
6. Đánh giá hiệu suất.....37
4. Qwen3 Đột phá kiến trúc..... 38
4.1. Đột phá Kiến trúc Cốt lõi: Dense và MoE Tiêu chuẩn..... 39
4.2. Cơ chế Hybrid Thinking và Thinking Budget.....41
4.3. Dữ liệu Siêu quy mô và Chiến lược Pre-training.....43
4.4. Post-training và Kỹ thuật Chưng cất (Distillation)..... 44
4.5. Đánh giá Hiệu năng, Scaling Laws và Năng lực Tiếng Việt.....45
5. Sự cải thiện của Qwen-VL qua từng phiên bản..... 47
5.1. Kiến trúc Qwen-VL Gốc và các Bottlenecks.....47
5.1.1 Kiến trúc Cơ sở và Giới hạn Độ phân giải Cố định.....47
5.1.2. Nút thắt Thông tin tại Khối Resampler.....48
5.2. Kiến trúc Qwen2-VL: Breaking the Mold..... 48
5.2.1. Naive Dynamic Resolution: Khả năng xử lý hình ảnh với độ phân giải động.....49
5.2.2. Loại bỏ Resampler: Kiến trúc MLP-based Merger tối giản và hiệu quả..... 50
5.2.3. Multimodal Rotary Position Embedding (M-RoPE): Tọa độ Hình học 1D–2D–3D..... 51
5.3. Kiến trúc Qwen2.5-VL: Hyper-speed processing..... 52

5.3.1. Cuộc cách mạng Attention: Window Attention trong ViT..... 53
5.3.2. Absolute Coordinate System và Dynamic FPS Training..... 54
5.3.3. Tinh chỉnh tăng cường đặc vụ: Visual-RFT và Visual-ARFT.. 55
5.4. Kiến trúc Qwen3-VL: Định hình lại Không gian Đa phương thức...56
5.4.1. Vision Encoder & Dynamic Resolution.....56
5.4.2. Vision-Language Merger & Cơ chế nén C2T..... 57
5.4.3. Đột phá Cơ chế Bơm Đặc trưng: DeepStack..... 58
5.4.4. 3D Grounding & Affordance Learning.....58
5.4.5. Multi-Image Understanding (Hiểu đa ảnh)..... 59
5.4.6. Xử lý Video Siêu dài với Interleaved MRoPE và Explicit Textual Timestamps..... 60
6. Qwen3.5 Kỷ nguyên AI Agents..... 61
6.1. Tổng quan về Sự ra mắt của Qwen3.5..... 61
6.2. Kiến trúc Hybrid Đột phá và Hiệu năng Tính toán..... 61
6.3. Năng lực Đa phương thức và Agent Thị giác (Visual Agents)..... 62
6.4. Khả năng Đa ngôn ngữ và Dữ liệu Tiền huấn luyện..... 63
6.5. Đánh giá Hiệu năng (Benchmarks)..... 63
6.6. Hạ tầng và Khung học tăng cường (RL Framework).....64
6.7. Các chế độ tương tác và Truy cập..... 64
6.8 Kết luận..... 65
IV. Xây dựng mô hình và thiết lập thực nghiệm.....65
1. Chuẩn bị và tiền xử lý dữ liệu..... 65
1.1. Giới thiệu về tập dữ liệu train MathVision..... 65
1.2 Giới thiệu về tập dữ liệu evaluate MathVista..... 67
2. Cấu trúc Prompt và Định dạng Dữ liệu..... 68
3. Kỹ thuật Finetuning.....71
3.1 Mô hình Cơ sở.....71
3.3 . Hàm Mất mát..... 72
4. Thiết lập tham số và môi trường.....73
4.1 Siêu tham số..... 73
4.2 Môi trường Huấn luyện.....74
V. Kết quả và đánh giá..... 75

5. Phương pháp và độ đo đánh giá..... 75
6. Kết quả định lượng..... 76
2.1 Đánh giá trên tập MathVision - testmini..... 76
2.2 Đánh giá trên tập MathVision - testmini..... 79
3. Đánh giá định tính..... 81
4. Phân tích lỗi..... 82

I.Giới thiệu thành viên và phân chia công việc

1. Giới thiệu thành viên

STT	MSSV	Họ tên
1	25C15031	Nguyễn Hoàng Anh
2	25C15032	Nguyễn Quốc Anh
3	25C15017	Lý Kim Long
4	25C15052	Nguyễn Đặng Nhật Minh
5	25C15047	Chu Hải Linh

2. Phân chia công việc

Các thành viên có đóng góp đồng đều vào việc thực hiện đồ án. Bảng dưới đây mô tả chi tiết các công việc được phân công thực hiện:

STT	Công việc	Thành viên đảm nhiệm	Phần trăm hoàn thành
1	Tìm hiểu mô hình Qwen 2, Qwen 3.5	Nguyễn Hoàng Anh	**100%**
2	Chạy thử mô hình trên bài toán thực tế	Nguyễn Quốc Anh	**100%**
3	Tìm hiểu mô hình Qwen 2.5, Qwen 2, 2.5 VL	Lý Kim Long	**100%**
4	Tìm hiểu mô hình Qwen 3, series Qwen VL từ 1 đến ver 3	Nguyễn Đặng Nhật Minh	**100%**
5	Tìm hiểu mô hình Qwen 1	Chu Hải Linh	**100%**

II.Giới thiệu tổng quan về đồ án

1. Bối cảnh nghiên cứu

Trong những năm gần đây, các mô hình ngôn ngữ lớn (Large Language Models – LLMs) dựa trên kiến trúc Transformer đã đạt nhiều tiến bộ trong các bài toán xử lý ngôn ngữ tự nhiên như sinh văn bản, hỏi đáp và hỗ trợ lập trình. Dòng mô hình Qwen là một trong những hệ LLM được phát triển theo hướng này, với các phiên bản Qwen-1, Qwen-2, Qwen-2.5 và Qwen-3 liên tục được cải tiến về kiến trúc và khả năng biểu diễn. Việc nghiên cứu các phiên bản này giúp hiểu rõ hơn cách một mô hình ngôn ngữ lớn được thiết kế, tối ưu và phát triển qua từng giai đoạn.

2. Mục tiêu nghiên cứu

Đồ án tập trung phân tích kiến trúc của các mô hình trong dòng Qwen từ Qwen-1 đến Qwen-3, trong đó làm rõ kiến trúc nền tảng của Qwen-1 và các cải tiến quan trọng ở các phiên bản sau. Bên cạnh phần phân tích lý thuyết, nhóm cũng thực hiện một số thực nghiệm nhằm quan sát và đánh giá ảnh hưởng của các thiết kế kiến trúc đối với hiệu năng của mô hình.

III. Kiến trúc và sự phát triển của Qwen

1. Qwen1 – Kiến trúc nền tảng

1. Tổng quan

Qwen-1 là mô hình ngôn ngữ lớn được phát triển bởi Alibaba Cloud, xây dựng dựa trên kiến trúc **Transformer dạng decoder-only** và hoạt động theo cơ chế **tự hồi quy (autoregressive)**, tức là dự đoán token tiếp theo dựa trên các token đã xuất hiện trước đó.

Trong quá trình xử lý, văn bản đầu vào được chuyển thành các **token**, sau đó ánh xạ thành **vector embedding** và đưa qua nhiều **Transformer decoder blocks**. Mỗi block bao gồm các thành phần chính như **multi-head self-attention**, **feed-forward network**, cùng với **normalization** và **residual connection** để học biểu diễn ngữ cảnh của chuỗi.

Qwen-1 được huấn luyện trên dữ liệu đa ngôn ngữ rất lớn với khoảng **3 nghìn tỷ token**. Tập dữ liệu huấn luyện được xây dựng từ nhiều nguồn khác nhau như **văn bản web, sách, bài báo, tài liệu học thuật và mã nguồn lập trình**, bao phủ nhiều lĩnh vực và nhiều ngôn ngữ, đặc biệt là tiếng Anh và tiếng Trung.

2. Tokenization và biểu diễn đầu vào

Tokenizer của Qwen

Trong mô hình Qwen-1, bước đầu tiên khi xử lý văn bản là tokenization, tức là chuyển văn bản thành các đơn vị nhỏ gọi là token. Qwen sử dụng một tokenizer được thiết kế riêng dựa trên phương pháp **Byte Pair Encoding (BPE).**

BPE là một phương pháp tokenization hoạt động theo hướng ghép dần các cặp ký tự xuất hiện nhiều trong dữ liệu để tạo thành các token mới. Ban đầu, văn bản được tách thành các ký tự riêng lẻ. Sau đó, thuật toán tìm các cặp ký tự xuất hiện nhiều nhất trong tập dữ liệu và gộp chúng lại thành một token mới. Quá trình này được lặp lại nhiều lần, dần dần hình thành các token có độ dài lớn hơn như một phần của từ hoặc toàn bộ từ.

Ví dụ đơn giản:

l o w

l o w e r

l o w e s t

Nếu cặp **"l"+"o"** xuất hiện nhiều, thuật toán sẽ gộp thành **"lo"**, sau đó tiếp tục ghép với các ký tự khác để tạo ra các token dài hơn như **"low"**.

Tokenizer của Qwen được thiết kế với **vocabulary lớn**, tối ưu cho các loại dữ liệu chính như **tiếng Anh, tiếng Trung và dữ liệu code**. Việc thiết kế vocabulary theo hướng này giúp mô hình biểu diễn tốt hơn các từ phổ biến trong ngôn ngữ tự nhiên cũng như các ký hiệu thường gặp trong lập trình.

Một điểm quan trọng của tokenizer là **ảnh hưởng trực tiếp đến độ dài chuỗi token (sequence length)**. Khi tokenizer có thể biểu diễn một từ hoặc một chuỗi ký tự bằng ít token hơn, chuỗi đầu vào của mô hình sẽ ngắn hơn. Điều này giúp giảm chi phí tính toán trong các lớp Transformer, đặc biệt vì cơ chế self-attention có độ phức tạp phụ thuộc vào độ dài chuỗi đầu vào.

Token Embedding

Sau khi quá trình tokenization hoàn thành, chuỗi token thu được sẽ được chuyển thành các vector số thông qua embedding layer của mô hình. Mỗi token được ánh xạ thành một vector có kích thước cố định theo công thức:

$$x_i = Embedding(token_i)$$

trong đó:

- $token_i$ là token thứ i trong chuỗi
- x_i là vector embedding tương ứng của token đó

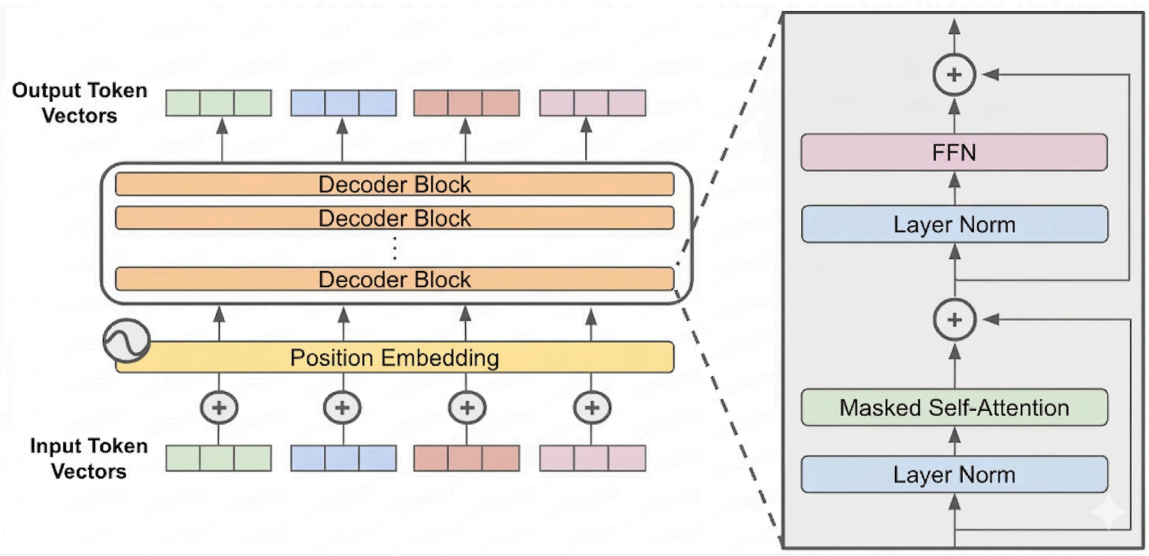
Tập hợp các vector này tạo thành biểu diễn đầu vào của chuỗi token:

$$X = (x_1, x_2, x_3, ..., x_n)$$

Mỗi vector embedding có kích thước d (embedding dimension), và kích thước này phụ thuộc vào cấu hình của từng phiên bản mô hình. Embedding dimension cũng chính là kích thước của **hidden representation** được sử dụng xuyên suốt trong các lớp Transformer.

Sau khi được ánh xạ thành vector embedding, toàn bộ chuỗi vector sẽ được đưa trực tiếp vào các lớp **Transformer decoder** của mô hình để thực hiện các phép tính attention và xây dựng biểu diễn ngữ cảnh cho chuỗi văn bản.

3. Kiến trúc tổng thể của Transformer trong Qwen



Hình 1.3.1 Kiến trúc tổng thể của Transformer decoder-only trong mô hình Qwen.

Mô hình được xây dựng dựa trên kiến trúc **Transformer** dạng decoder-only, trong đó các khối **Decoder Block** được xếp chồng nhiều tầng để xử lý chuỗi token đầu vào và học biểu diễn ngữ nghĩa của văn bản.

Đầu vào của mô hình là chuỗi **input token vectors**, được tạo ra bằng cách ánh xạ các token văn bản sang vector embedding. Sau đó, thông tin vị trí của từng token trong chuỗi được đưa vào thông qua **position embedding** nhằm giúp mô hình nhận biết thứ tự của các token trong câu. Vector biểu diễn của mỗi token sau bước này được sử dụng làm đầu vào cho các **decoder blocks** của Transformer.

Các **decoder blocks** được xếp chồng thành nhiều tầng, mỗi tầng thực hiện hai phép biến đổi chính gồm **masked self-attention** và **feed-forward neural network (FFN)**. Cơ chế masked self-attention cho phép mỗi token chỉ có thể truy cập thông tin từ các token đứng trước nó trong chuỗi, nhờ đó

mô hình có thể thực hiện quá trình sinh văn bản theo cách **tự hồi quy (autoregressive)**.

Nếu ký hiệu ma trận biểu diễn đầu vào của một block là X, quá trình tính toán trong block có thể được mô tả như sau:

$$H_1 = X + \text{Attention}\big(\text{Norm}(X)\big)$$

Trong đó $Attention(.)$ biểu diễn phép biến đổi của masked self-attention và $Norm(.)$ là lớp chuẩn hóa. Kết quả của attention được cộng với đầu vào ban đầu thông qua **residual connection** nhằm giữ lại thông tin của biểu diễn trước đó.

Sau bước attention, dữ liệu tiếp tục được chuẩn hóa và đưa vào mạng feed-forward:

$$H_2 = H_1 + \text{FFN}\big(\text{Norm}(H_1)\big)$$

Trong đó $FFN(.)$ là mạng feed-forward được áp dụng độc lập cho từng token nhằm tăng khả năng biểu diễn phi tuyến của mô hình.

Các decoder blocks được lặp lại nhiều tầng để học các mức biểu diễn ngữ nghĩa khác nhau của chuỗi văn bản. Nếu mô hình có L tầng Transformer, biểu diễn của chuỗi token sau mỗi tầng được tính như sau:

$$H^{(l+1)} = \text{TransformerBlock}\big(H^{(l)}\big)$$

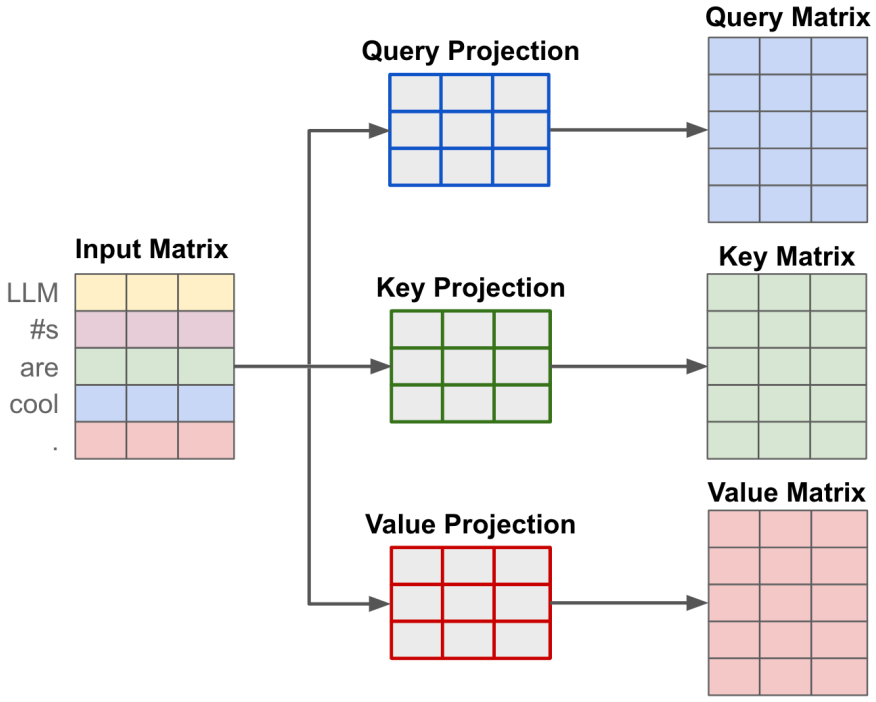
với:

$$H^{(0)} = \text{Embedding}(X)$$

Sau khi đi qua toàn bộ các tầng Transformer, các vector biểu diễn cuối cùng của chuỗi token được sử dụng để tạo ra **output token vectors**, từ đó mô hình tính phân phối xác suất của token tiếp theo trong quá trình sinh văn bản.

4. Cơ chế Self-Attention

Trong kiến trúc Transformer của Qwen, cơ chế **self-attention** cho phép mỗi token trong chuỗi học được mối quan hệ với các token khác trong cùng chuỗi. Nhờ đó mô hình có thể khai thác thông tin ngữ cảnh của toàn bộ câu thay vì chỉ các token lân cận.



Hình 1.4.1 Quá trình tạo Query, Key và Value trong cơ chế Self-Attention của Transformer.

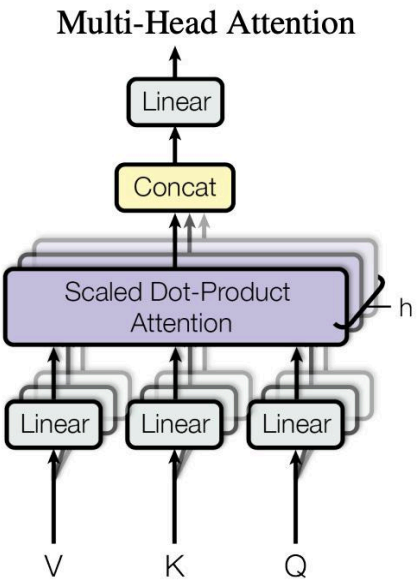
Giả sử đầu vào của tầng attention là ma trận biểu diễn token X. Từ đó ba ma trận **Query**, **Key** và **Value** được tạo ra thông qua các phép biến đổi tuyến tính:

$$Q = XW_Q \qquad K = XW_K \qquad V = XW_V$$

Cơ chế attention được tính bằng cách đo mức độ tương quan giữa các vector Query và Key, sau đó chuẩn hóa bằng hàm softmax để tạo trọng số attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Trong mô hình Qwen, attention được áp dụng dưới dạng **masked self-attention**, trong đó mỗi token chỉ có thể truy cập thông tin từ các token đứng trước nó trong chuỗi. Điều này đảm bảo rằng mô hình hoạt động theo cơ chế **tự hồi quy (autoregressive)** khi dự đoán token tiếp theo.



Hình 1.4.2 Cơ chế Multi-Head Attention trong Transformer

Ngoài ra, Transformer sử dụng **multi-head attention**, trong đó phép tính attention được thực hiện song song trên nhiều không gian biểu diễn khác nhau. Kết quả của các attention head sau đó được nối lại để tạo thành biểu diễn cuối cùng:

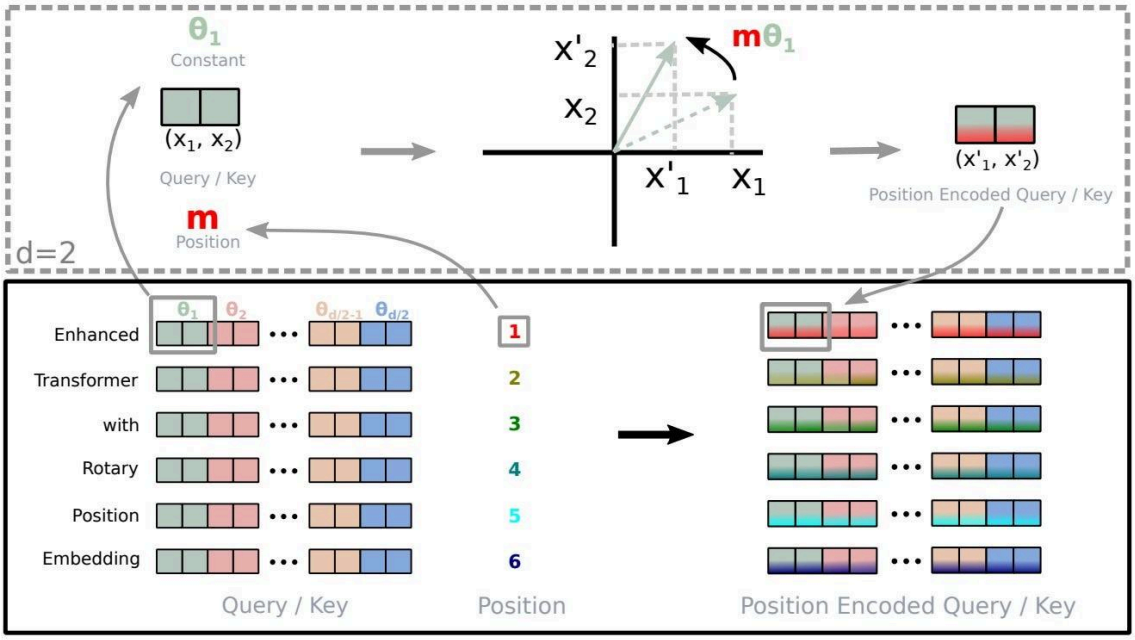
$$\text{MultiHead}(Q, K, V) = \text{Concat}(head_1, \dots, head_h)W_O$$

Cơ chế này giúp mô hình học được nhiều loại quan hệ khác nhau giữa các token trong chuỗi văn bản.

5. Positional Encoding

Trong kiến trúc Transformer, cơ chế self-attention không tự chứa thông tin về thứ tự của các token trong chuỗi. Vì vậy, mô hình cần một phương pháp

bổ sung để biểu diễn vị trí của từng token, được gọi là **positional encoding**. Trong mô hình Qwen, phương pháp được sử dụng là **Rotary Positional Embedding (RoPE)**.



Hình 1.5.1 Cơ chế Rotary Positional Embedding (RoPE) dùng để mã hóa thông tin vị trí trong Transformer.

RoPE biểu diễn thông tin vị trí bằng cách áp dụng phép **quay (rotation)** lên các vector biểu diễn của token trong không gian embedding. Thay vì cộng trực tiếp vector vị trí vào embedding của token như các phương pháp positional encoding truyền thống, RoPE tích hợp thông tin vị trí trực tiếp vào quá trình tính toán attention. Cách tiếp cận này giúp mô hình nắm bắt tốt hơn quan hệ vị trí tương đối giữa các token trong chuỗi.

Giả sử vector embedding của một token được chia thành các cặp thành phần

(x_{2i}, x_{2i+1}) . Trong RoPE, mỗi cặp này được biến đổi thông qua phép quay trong mặt phẳng hai chiều:

$$(x'_{2i} x'_{2i+1}) = \begin{pmatrix} \cos\theta_i & -\sin\theta_i \\ \sin\theta_i & \cos\theta_i \end{pmatrix} (x_{2i} x_{2i+1})$$

Trong đó θ_i là góc quay phụ thuộc vào vị trí của token trong chuỗi. Nhờ phép biến đổi này, thông tin vị trí được mã hóa trực tiếp vào vector biểu diễn trước khi thực hiện phép tính attention.

Việc sử dụng RoPE mang lại một số ưu điểm quan trọng. Thứ nhất, phương pháp này cho phép mô hình biểu diễn **quan hệ vị trí tương đối giữa các token**, điều này đặc biệt hữu ích trong các chuỗi văn bản dài. Thứ hai, RoPE có khả năng **mở rộng tốt sang các chuỗi dài hơn so với độ dài huấn luyện**, giúp cải thiện khả năng xử lý ngữ cảnh dài của mô hình.

Nhờ tích hợp thông tin vị trí trực tiếp vào cơ chế attention, RoPE trở thành một thành phần quan trọng trong kiến trúc Transformer của Qwen, giúp mô hình hiểu được cấu trúc và thứ tự của các token trong chuỗi văn bản.

6. Normalization

Trong các mô hình Transformer lớn, **normalization** đóng vai trò quan trọng để ổn định quá trình huấn luyện. Nếu không có bước chuẩn hóa, giá trị của các vector trong mạng có thể tăng hoặc giảm quá mạnh qua nhiều lớp, dẫn đến hiện tượng **gradient không ổn định**. Trong kiến trúc Qwen-1, hai thiết kế chính liên quan đến normalization là **RMSNorm** và **pre-normalization architecture**.

RMSNorm

Thay vì sử dụng LayerNorm như trong Transformer gốc, Qwen sử dụng **Root Mean Square Normalization (RMSNorm)**. Ý tưởng của RMSNorm là chuẩn hóa vector đầu vào dựa trên độ lớn trung bình bình phương của các phần tử trong vector.

Công thức của RMSNorm:

$$\mathrm{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{n}\sum_{i=1}^{n} x_i^2 + \epsilon}} \cdot g$$

Trong đó:

- $x = (x_1, x_2, x_3, ..., x_n)$ là vector đầu vào
- n là số chiều của vector
- ϵ là một hằng số nhỏ để tránh chia cho 0
- g là **tham số scale học được** của mô hình

Cách hoạt động có thể hiểu đơn giản như sau: đầu tiên tính **root mean square (RMS)** của vector đầu vào, sau đó chia toàn bộ vector cho giá trị này để đưa độ lớn của vector về một thang đo ổn định. Cuối cùng, vector được nhân với tham số g để mô hình có thể điều chỉnh lại độ lớn nếu cần.

So với **LayerNorm**, RMSNorm **không trừ đi giá trị trung bình (mean)** của vector mà chỉ chuẩn hóa theo độ lớn. Vì vậy phép tính đơn giản hơn và giảm chi phí tính toán. Trong các mô hình lớn, cách chuẩn hóa này vẫn giữ

được hiệu quả ổn định trong huấn luyện, đồng thời giúp mô hình chạy nhanh hơn.

Pre-Normalization Architecture

Ngoài việc sử dụng RMSNorm, Qwen còn áp dụng **pre-normalization architecture** trong mỗi Transformer block. Điều này nghĩa là bước normalization được thực hiện trước các phép biến đổi chính như attention hoặc feed-forward.

Thứ tự xử lý trong một block có dạng:

- Norm → Attention → Residual
- Norm → FFN → Residual

Có thể biểu diễn đơn giản như sau:

$$h_1 = x + Attention(RMSNorm(x))$$

$$h_2 = h_1 + FFN(RMSNorm(h_1))$$

Trong đó:

- x là đầu vào của block
- h_1 là kết quả sau attention
- h_2 là đầu ra cuối của block

Cách thiết kế này giúp **gradient truyền qua nhiều lớp ổn định hơn** so với kiến trúc post-normalization trong Transformer gốc. Điều này đặc biệt quan

trọng khi huấn luyện các mô hình có **rất nhiều lớp và số tham số lớn**, vì nó giúp giảm nguy cơ gradient bị biến mất hoặc bùng nổ trong quá trình huấn luyện.

7. Feed Forward Network

Sau khi mỗi token đi qua **self-attention**, bước tiếp theo trong Transformer block là **Feed Forward Network (FFN)**. Thành phần này hoạt động độc lập trên từng token, tức là mỗi vector biểu diễn token được xử lý riêng qua một mạng neural nhỏ gồm các lớp tuyến tính và hàm kích hoạt phi tuyến.

Kiến trúc FFN

Trong Transformer cơ bản, FFN thường gồm hai phép biến đổi tuyến tính với một hàm kích hoạt phi tuyến ở giữa. Công thức tổng quát:

$$FFN(x) = max(0, xW_1 + b_1)W_2 + b_2$$

Trong đó:

- x là vector biểu diễn của token
- W_1, W_2 là các ma trận trọng số
- b_1, b_2 là các vector bias
- $max(0, \cdot)$ tương ứng với hàm kích hoạt **ReLU**

Ý tưởng của FFN là **mở rộng không gian biểu diễn của token**, cho phép mô hình học được các quan hệ phi tuyến phức tạp hơn sau khi attention đã tổng hợp thông tin ngữ cảnh.

SwiGLU Activation

Trong Qwen-1, FFN không sử dụng ReLU như Transformer ban đầu mà sử dụng SwiGLU activation. Đây là một biến thể của Gated Linear Unit (GLU), trong đó một nhánh của mạng đóng vai trò như cổng (gate) để điều chỉnh thông tin từ nhánh còn lại.

Cấu trúc của FFN trong Qwen có dạng:

$$FFN(x) = (xW_1) \otimes Swish(xW_2)$$

Trong đó:

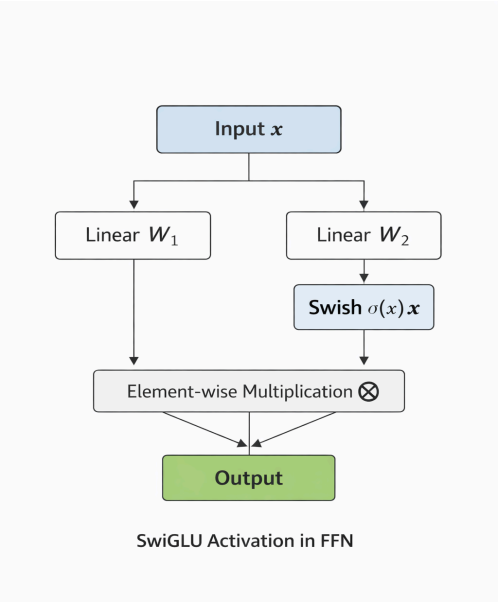
- $\otimes$ là phép nhân từng phần tử (element-wise multiplication)
- $Swish(.)$ là hàm kích hoạt được định nghĩa bởi:

$$Swish(x) = x\sigma(x)$$

với $\sigma(x)$ là hàm sigmoid.

Cơ chế hoạt động có thể hiểu như sau:

- nhánh xW_1 tạo ra thông tin đặc trưng
- nhánh $Swish(xW_2)$ đóng vai trò cổng điều khiển (gate)
- phép nhân từng phần tử giúp mô hình chọn lọc thông tin quan trọng trước khi đưa sang lớp tiếp theo.



Hình 1.7.1 Sơ đồ minh họa SwiGLU

Nhờ cơ chế **gating**, SwiGLU giúp mô hình học được các biểu diễn **phi tuyến linh hoạt hơn** so với các hàm kích hoạt đơn giản như ReLU hoặc GeLU. Điều này giúp cải thiện khả năng biểu diễn của mạng FFN trong các mô hình ngôn ngữ lớn.

8. Embedding và Output Layer

Trong mô hình Qwen, lớp **embedding** đóng vai trò chuyển đổi các token văn bản thành các vector số học để mô hình có thể xử lý. Sau khi văn bản đầu vào được chuyển thành chuỗi token thông qua tokenizer, mỗi token sẽ được ánh xạ sang một vector có kích thước cố định trong không gian embedding. Các vector này tạo thành ma trận biểu diễn đầu vào cho mô hình Transformer.

Giả sử chuỗi token đầu vào được ký hiệu là $x_1, x_2, ..., x_n$. Mỗi token được ánh xạ sang vector embedding thông qua ma trận embedding E:

$$e_i = E(x_i)$$

Trong đó e_i là vector embedding của token x_i, còn E là ma trận tham số được học trong quá trình huấn luyện. Tập hợp các vector embedding này tạo thành đầu vào cho các tầng Transformer của mô hình.

Sau khi chuỗi token đi qua toàn bộ các tầng Transformer, mô hình thu được các vector biểu diễn ẩn cho từng token. Để dự đoán token tiếp theo trong quá trình sinh văn bản, các vector này được đưa qua một **lớp linear projection** để chuyển từ không gian biểu diễn ẩn sang không gian của vocabulary. Sau đó hàm softmax được áp dụng để tạo phân phối xác suất cho tất cả các token trong từ vựng:

$$P(x_{t+1}) = \text{softmax}(h_t W_o)$$

Trong đó h_t là vector biểu diễn của token cuối cùng sau các tầng Transformer và W_o là ma trận trọng số của lớp chiếu đầu ra. Kết quả của hàm softmax là xác suất xuất hiện của từng token trong vocabulary, từ đó mô hình có thể chọn token tiếp theo trong chuỗi.

Nhờ sự kết hợp giữa lớp embedding ở đầu vào và lớp chiếu đầu ra ở cuối mô hình, Qwen có thể chuyển đổi qua lại giữa biểu diễn văn bản rời rạc và biểu diễn vector liên tục trong quá trình xử lý và sinh ngôn ngữ tự nhiên.

9. Cơ chế mở rộng context trong kiến trúc

Trong các mô hình Transformer, độ dài ngữ cảnh (context length) của cơ chế attention bị giới hạn bởi chi phí tính toán. Khi số lượng token trong chuỗi đầu vào tăng lên, mỗi token phải tính toán attention với tất cả các token còn lại. Do đó độ phức tạp tính toán của cơ chế self-attention tăng theo bậc hai của chiều dài chuỗi:

$$Complexity = O(n^2)$$

Trong đó n là số lượng token trong chuỗi đầu vào. Khi context length tăng, chi phí tính toán và bộ nhớ cần thiết cho ma trận attention cũng tăng nhanh, gây khó khăn cho việc xử lý các chuỗi văn bản dài.

Để mở rộng khả năng xử lý ngữ cảnh dài, Qwen áp dụng một số kỹ thuật **training-free** trong giai đoạn suy luận. Một kỹ thuật quan trọng là **NTK-aware interpolation**, trong đó tham số cơ sở của Rotary Positional Embedding (RoPE) được điều chỉnh nhằm giảm mất mát thông tin tần số cao khi mở rộng độ dài chuỗi. Ngoài ra, **dynamic NTK-aware interpolation** được sử dụng để thay đổi hệ số scale theo từng đoạn của chuỗi, giúp hạn chế sự suy giảm hiệu năng khi context length tăng.

Bên cạnh đó, Qwen còn sử dụng **LogN-Scaling** để điều chỉnh giá trị dot-product trong attention theo tỉ lệ giữa độ dài ngữ cảnh hiện tại và độ dài huấn luyện, giúp ổn định phân bố attention khi chuỗi dài hơn. Mô hình cũng áp dụng **window attention** để giới hạn phạm vi attention trong một cửa sổ ngữ cảnh nhất định. Ngoài ra, các tầng trong mô hình sử dụng kích thước

cửa sổ khác nhau, với cửa sổ nhỏ hơn ở các tầng thấp và lớn hơn ở các tầng cao nhằm tối ưu khả năng xử lý chuỗi dài.

2. Qwen2 Bước tiến quan trọng

1. Tổng quan

Qwen2 là thế hệ kế tiếp sau Qwen 1.5 trong dòng mô hình ngôn ngữ lớn và mô hình đa phương thức lớn của Alibaba. Đây là một bộ sưu tập toàn diện các mô hình ngôn ngữ từ nền tảng (base) đến tinh chỉnh theo hướng dẫn (instruction-tuned), với quy mô tham số đa dạng từ 0,5 tỷ đến 72 tỷ. Qwen2 không chỉ vượt qua dòng Qwen1.5 tiền nhiệm mà còn thể hiện hiệu năng cạnh tranh mạnh mẽ với các mô hình độc quyền hàng đầu trong các bài thử nghiệm về hiểu ngôn ngữ, toán học, lập trình và suy luận.

2. Bộ mã hóa token & Mô hình

a) Bộ mã hóa token

- Qwen2 tiếp tục sử dụng bộ mã hóa dựa trên byte-level byte-pair encoding (BPE) tương tự như Qwen1.
- Bộ mã hóa này có hiệu suất nén cao, giúp tăng cường khả năng đa ngôn ngữ và hiệu quả tính toán.
- Kích thước từ vựng: Bao gồm 151.643 token thường và 3 token điều khiển.

b) Kiến trúc mô hình

Qwen2 được xây dựng dựa trên kiến trúc Transformer với cơ chế tự chú ý nhân quả (causal masks).

Mô hình dày đặc Qwen2 (Qwen2 Dense Model):

Grouped Query Attention (GQA): Thay thế cho Multi-head Attention truyền thống để tối ưu hóa việc sử dụng bộ nhớ KV cache, từ đó tăng tốc độ xử lý (throughput) khi suy luận.

Dual Chunk Attention (DCA) & YARN: Giúp mở rộng cửa sổ ngữ cảnh và cải thiện khả năng nắm bắt thông tin vị trí tương đối giữa các token trong các chuỗi văn bản dài.

Các thành phần khác: Sử dụng kích hoạt SwiGLU, mã hóa vị trí RoPE, và RMSNorm để đảm bảo sự ổn định khi huấn luyện.

Mô hình Hỗn Hợp Chuyên Gia Qwen2 (Qwen2 Mixture-of-Experts)

Mô hình **Qwen2 Mixture-of-Experts (MoE)**, cụ thể là phiên bản **Qwen2-57B-A14B**, là một bước tiến quan trọng trong việc tối ưu hóa hiệu suất tính toán bằng cách sử dụng kiến trúc thưa (sparsity) thay vì kiến trúc dày đặc truyền thống,.

Dưới đây là chi tiết về kiến trúc, công thức và các cải tiến đặc biệt của mô hình này:

Nguyên lý hoạt động và Công thức

Thay vì sử dụng một mạng Feed-Forward (FFN) duy nhất cho mọi token, kiến trúc MoE thay thế lớp FFN truyền thống bằng nhiều lớp FFN riêng biệt, mỗi lớp đóng vai trò là một **"chuyên gia" (expert)**.

Quá trình xử lý một token đầu vào (x) được thực hiện qua hai bước chính:

- **Mạng cổng (Gating Network):** Xác định xác suất token x sẽ được gửi đến chuyên gia nào. Công thức tính xác suất (p) là:

$$p = softmax(G(x))$$

Trong đó $G(x)$ là đầu ra của mạng cổng.

- **Tổng hợp kết quả:** Chỉ một số lượng chuyên gia nhất định ($top\ k$) có xác suất cao nhất được kích hoạt để tính toán cho token đó,. Kết quả cuối cùng (y) là tổng trọng số từ các chuyên gia được chọn:

$$y = \sum_{i \in top\ k(p)} p_i E_i(x)$$

Trong đó, $E_i(x)$ là kết quả tính toán của chuyên gia thứ i.

Các cải tiến cốt lõi trong Qwen2 MoE

- **Độ mịn của chuyên gia (Expert Granularity):** Khác với các mô hình MoE thông thường chỉ đơn giản là chia nhỏ một FFN lớn, Qwen2 sử dụng các **chuyên gia mịn (fine-grained experts)**. Bằng cách tạo ra nhiều chuyên gia có kích thước nhỏ hơn và kích hoạt nhiều chuyên gia cùng lúc, mô hình tạo ra được nhiều sự kết hợp chuyên gia đa dạng và linh hoạt hơn, giúp tăng cường khả năng thích ứng với các tác vụ khác nhau.
- **Định tuyến chuyên gia (Expert Routing):** Mô hình tích hợp cả **chuyên gia chia sẻ (shared experts)** và **chuyên gia điều hướng (routed experts)**.
    - **Shared experts:** Luôn được kích hoạt để xử lý các kiến thức chung cần thiết cho mọi tác vụ.

    - **Routed experts:** Chỉ được chọn lọc dựa trên mạng cổng để xử lý các khía cạnh chuyên biệt.
- **Khởi tạo chuyên gia (Expert Initialization):** Qwen2 áp dụng phương pháp tương tự như "upcycling", khởi tạo các chuyên gia từ trọng số của mô hình dày đặc. Để tăng tính đa dạng, các tham số được xáo trộn dọc theo chiều trung gian và 50% tham số của mỗi chuyên gia mịn được khởi tạo ngẫu nhiên lại để thúc đẩy khả năng học hỏi trong quá trình huấn luyện.

Cấu hình chi tiết (Qwen2-57B-A14B)

Dựa trên cấu hình trong các nguồn tài liệu, mô hình MoE của Qwen2 có các thông số ấn tượng:

- **Tổng tham số:** 57 tỷ (57B), nhưng chỉ có **14 tỷ (14B) tham số được kích hoạt** cho mỗi token,.
- **Số lượng chuyên gia:** Gồm **64 chuyên gia điều hướng** và **8 chuyên gia chia sẻ**.
- **Số chuyên gia kích hoạt:** Đối với mỗi token, ngoài các chuyên gia chia sẻ, có **8 chuyên gia điều hướng** được chọn để tính toán.
- **Hiệu quả:** Mô hình này đạt được hiệu suất tương đương với các mô hình dày đặc có quy mô 30 tỷ tham số nhưng với chi phí tính toán thấp hơn nhiều.

Qwen2-57B-A14B đặc biệt mạnh trong các tác vụ liên quan đến **lập trình (coding)** và **toán học (mathematics)** so với các đối thủ cùng phân khúc,.

2.2.3 Cấu hình mô hình

Qwen2 cung cấp 5 kích thước mô hình chính: 0.5B, 1.5B, 7B, 57B-A14B (MoE) và 72B. Đáng chú ý, mô hình MoE 57B chỉ kích hoạt 14B tham số cho mỗi token nhưng đạt hiệu suất tương đương với các mô hình dày đặc 30B.

Kiến trúc của các mô hình Qwen2 dày đặc (dense) và MoE

Cấu hình	0.5B	1.5B	7B	72B	57B-A14B (MoE)
Kích thước ẩn (Hidden Size)	896	1,536	3,584	8,192	3,584
Số lớp (# Layers)	24	28	28	80	28
Số đầu truy vấn (# Query Heads)	14	12	28	64	28
Số đầu KV (# KV Heads)	2	2	4	8	4
Kích thước đầu (Head Size)	64	128	128	128	128
Kích thước trung gian (Intermediate Size)	4,864	8,960	18,944	29,568	2,560

Số chuyên gia điều hướng (# Routed Experts)	-	-	-	-	64
Số chuyên gia kích hoạt (# Activated Experts)	-	-	-	-	8
Số chuyên gia chia sẻ (# Shared Experts)	-	-	-	-	8
Ràng buộc Embedding (Embedding Tying)	True	True	False	False	False
Kích thước từ vựng (Vocabulary Size)	151,646	151,646	151,646	151,646	151,646
Số lượng token huấn luyện (# Trained Tokens)	12T	7T	7T	7T	4.5T

2.3. Tiền huấn luyện

2.3.1 Dữ liệu tiền huấn luyện

Quy mô: Được huấn luyện trên tập dữ liệu đa ngôn ngữ khổng lồ lên tới 7 nghìn tỷ (trillion) tokens (riêng mẫu 0.5B sử dụng 12 nghìn tỷ tokens).

Chất lượng: Thuật toán lọc dữ liệu được cải tiến bằng các phương pháp heuristic và dựa trên mô hình (sử dụng chính các mẫu Qwen trước đó để lọc và tổng hợp dữ liệu chất lượng cao).

Đa dạng: Hỗ trợ khoảng 30 ngôn ngữ, bao gồm tiếng Anh, tiếng Trung, tiếng Việt, tiếng Pháp, tiếng Đức, tiếng Nhật....

2.3.2 Huấn luyện ngữ cảnh dài

Trong giai đoạn cuối của tiền huấn luyện, độ dài ngữ cảnh được nâng từ 4.096 lên 32.768 tokens.

Tần số cơ sở của RoPE được điều chỉnh từ 10.000 lên 1.000.000 để tối ưu hóa hiệu suất trong các kịch bản ngữ cảnh dài.

2.4. Hậu huấn luyện

2.4.1 Dữ liệu hậu huấn luyện

Quy trình hậu huấn luyện tập trung vào việc căn chỉnh mô hình với sự giám sát tối thiểu của con người nhưng vẫn đảm bảo chất lượng dữ liệu cao.

Chú thích dữ liệu cộng tác

Sử dụng công cụ InsTag để trích xuất kho ngữ liệu lệnh (instruction ontology).

Áp dụng chiến lược tự tiến hóa (self-evolution) để yêu cầu mô hình Qwen tự thêm các ràng buộc phức tạp vào các lệnh hiện có.

Tổng hợp dữ liệu tự động

Rejection Sampling: Sử dụng cho các bài toán toán học để giữ lại các lộ trình suy luận dẫn đến câu trả lời đúng.

Execution Feedback: Sử dụng cho lập trình, mô hình tự tạo mã và các bộ kiểm thử để xác minh độ chính xác qua phản hồi thực thi.

Data Repurposing: Chuyển đổi các tác phẩm văn học công cộng thành dữ liệu huấn luyện cho các tác vụ sáng tạo và nhập vai.

2.4.2 Tinh chỉnh có giám sát (SFT)

Mô hình được tinh chỉnh trên hơn 500.000 mẫu ví dụ bao gồm các kỹ năng từ hiểu lệnh, lập trình đến an toàn thông tin.

2.4.3 Học tăng cường từ phản hồi con người (RLHF)

Giai đoạn ngoại tuyến: Sử dụng phương pháp Direct Preference Optimization (DPO) trên dữ liệu ưu tiên đã biên soạn sẵn.

Giai đoạn trực tuyến: Mô hình tự tinh chỉnh trong thời gian thực thông qua mô hình phần thưởng (reward model) để nhận phản hồi tức thì.

2.5. Đánh giá

2.5.1 Mô hình ngôn ngữ nền tảng

Năng lực cốt lõi

Qwen2-72B vượt qua Llama-3-70B trong các bài kiểm tra kiến thức chung (MMLU), khoa học (GPQA) và đặc biệt vượt trội trong toán học/lập trình.

Các mô hình nhỏ (0.5B, 1.5B) vẫn duy trì khả năng toán học và suy luận tốt so với các đối thủ cùng tầm như Phi-2 hay Gemma-2B.

2.5.2 Mô hình được tinh chỉnh theo hướng dẫn

Đánh giá trên bộ kiểm tra mở

Qwen2-72B-Instruct dẫn đầu trên các bảng xếp hạng như MT-Bench (9.12) và Arena-Hard (48.1), cho thấy khả năng căn chỉnh ưu tiên con người xuất sắc.

Đánh giá tự động nội bộ

Trong các bài đánh giá tiếng Trung nội bộ, Qwen2-72B-Instruct vượt qua cả Qwen1.5-110B dù có ít tham số hơn.

Khả năng ngữ cảnh dài

Thông qua các bài kiểm tra "Needle in a Haystack", Qwen2-72B cho thấy độ chính xác tuyệt đối trong việc truy xuất thông tin ở ngữ cảnh lên tới 128.000 tokens.

Đánh giá đa ngôn ngữ

Đánh giá bởi chuyên gia trên hơn 10 ngôn ngữ cho thấy Qwen2-72B-Instruct vượt trội so với GPT-3.5-Turbo và cạnh tranh trực tiếp với GPT-4-Turbo.

An toàn & Trách nhiệm

Qwen2 thể hiện khả năng từ chối các yêu cầu độc hại (lừa đảo, khiêu dâm, quyền riêng tư) tốt hơn so với GPT-4 và Mixtral-8x22B.

Phân tích nhiễm dữ liệu

Đội ngũ phát triển đã thực hiện phân tích kỹ lưỡng để loại bỏ các mẫu thử nghiệm khỏi dữ liệu huấn luyện, đảm bảo kết quả đánh giá là khách quan và trung thực.

2.6 Kết luận

Dòng mô hình Qwen2 đánh dấu một bước tiến quan trọng với sự kết hợp giữa kiến trúc dày đặc và MoE, khả năng xử lý ngữ cảnh cực dài và hiệu suất đa ngôn ngữ vượt trội. Việc mở nguồn các trọng số mô hình giúp thúc đẩy sự đổi mới trong cộng đồng nghiên cứu AI toàn cầu

3. Qwen2.5 - Cải thiện đáng kể

1. Tổng quan

Qwen 2.5 là một loạt các mô hình ngôn ngữ lớn được thiết kế để đáp ứng các nhu cầu đa dạng. So với các phiên bản trước, Qwen 2.5 đã được cải thiện đáng kể trong cả giai đoạn tiền huấn luyện (pre-training) và sau huấn luyện

(post-training). Bằng việc kết hợp giữa mở rộng quy mô dữ liệu và dữ liệu đa dạng, năng lực mô hình Qwen 2.5 đã nâng cao và thậm chí ở 1 số bài kiểm tra đã cho kết quả vượt qua cả mô hình Llama 405B.

2. Những cải tiến chính

- Kích thước đa dạng hơn: ngoài các size 0.5B, 1.5B, 7B, 72B, Qwen 2.5 có thêm các kích thước 3B, 14B và 32B. Các kích thước này tối ưu về chi phí cho các trường hợp hạn chế về tài nguyên. Trong đó, Qwen 2.5-Turbo và Qwen 2.5-Plus cung cấp sự cân bằng giữa độ chính xác, độ trễ và chi phí.
- Dữ liệu chất lượng hơn: dữ liệu pre-training đã tăng từ 7 nghìn tỷ lên hơn 18 nghìn tỷ token, tập trung vào kiến thức tổng quát, lập trình và toán học.
- Sử dụng hiệu quả hơn: Các giới hạn của Qwen 2 đã được loại bỏ:
    - Độ dài tạo văn bản tăng từ 2K lên hơn 8K token.
    - Hỗ trợ tốt hơn cho dữ liệu có cấu trúc (bảng biểu, JSON) và sử dụng công cụ.
    - Phiên bản Qwen 2.5-Turbo hỗ trợ độ dài ngữ cảnh lên đến 1 triệu token.

3. Kiến trúc và bộ mã hóa (architecture & tokenizer)

- Kiến trúc: tương tự Qwen 2, Qwen 2.5 duy trì kiến trúc Transformer-based decoder với các thành phần như Grouped Query Attention để tối ưu KV cache, cũng như hàm kích hoạt SwiGLU và

Rotary Positional Embeddings (RoPE), Các mô hình Mixture-of-Experts như Qwen 2.5-Turbo thay thế các lớp feed-forward network tiêu chuẩn bằng các lớp Mixture-of-Experts chuyên biệt.

- Tokenizer: Qwen 2.5 sử dụng bộ mã hóa byte-level Byte pair encoding với từ vựng gồm 151643 token. Ngoài ra, số lượng control token tăng từ 3 lên 22 so với Qwen 2, giúp thống nhất và tăng cường sự ổn định của mô hình.

4. Tiền huấn luyện (pre-training)

Gồm 4 khía cạnh:

- Lọc dữ liệu tốt hơn: các mô hình Qwen 2-Instruct được sử dụng làm bộ lọc để phân tích, đánh giá mẫu huấn luyên một cách đa chiều. Nhờ giai đoạn xử lý bằng các mô hình Qwen 2, dữ liệu giờ đây có chất lượng cao hơn và loại bỏ các mẫu chất lượng thấp.
- Dữ liệu toán và coding tốt hơn: các mô hình Qwen 2.5 được tích hợp dữ liệu từ các mô hình chuyên biệt là Qwen 2.5-Math và Qwen 2.5-Coder. Điều này giúp các mô hình này kế thừa khả năng tóan và coding mạnh mẽ từ các bộ dữ liệu chất lượng cao chuyên biệt.
- Dữ liệu tổng hợp tốt hơn: sử dụng các mô hình Qwen 2 72B để tạo dữ liệu toán học và lập trình chất lượng cao và lọc lại bằng các reward model để tăng cường dữ liệu hơn.
- Tỷ lệ giữa các thành phần trong dữ liệu tốt hơn: điều chỉnh lại tỷ lệ các domain trong bộ dữ liệu để giảm bớt các nội dung lặp lại từ web như giải trí, mạng xã hội và tăng cường các nội dung hàm lượng tri thức cao về khoa học, công nghệ.

5. Sau huấn luyện (post-training)

Gồm 2 bước tiến lớn:

- Mở rộng độ bao phủ dữ liệu cho supervised finetuning SFT: Qwen 2.5 sử dụng hơn 1 triệu mẫu dữ liệu chất lượng cao. Điều này giúp cải thiện các yếu điểm của mô hình Qwen 2, ví dụ như suy luận logic, giải toán, lập trình, hiểu dữ liệu có cấu trúc.
- Học tăng cường 2 giai đoạn:
    - Offline RL: tập trung vào phát triển các kỹ năng khó đánh giá bằng reward model như suy luận, coding, toán, instruction following (làm theo hướng dẫn).
    - Online RL: sử dụng reward model để tinh chỉnh các mặt như tính hữu ích, sự ngắn gọn và tính an toàn, giúp các output của model đáp ứng tiêu chuẩn của con người.

Chi tiết về OfflineRL:

- Phương pháp tạo dữ liệu: dùng chính mô hình đã qua giai đoạn tinh chỉnh để tạo ra các phản hồi (response) mới cho tập hợp câu hỏi mới (dữ liệu mới).
- Đánh giá chất lượng dữ liệu mới:
    - Execution feedback: đối với mã nguồn, các đoạn code được chạy trong môi trường sandbox xem có hoạt động đúng như yêu cầu không.
    - Answer matching (khớp đáp án): đối với toán, kết quả của mô hình được đối chiếu với đáp án chuẩn.
    - Phân loại: positive với những response vượt qua kiểm tra về chất lượng, negative với các response còn lại.

    - Ngoài ra, quy trình đánh giá còn kết hợp thêm rà soát bởi con người.
- Quy mô: quy trình tạo ra khoảng 150.000 cặp huấn luyện, giúp mô hình học được các kỹ năng phức tạp thông qua việc phân biệt được cách làm đúng và sai trong thực tế.

Chi tiết về OnlineRL: Qwen 2.5 sử dụng thuật toán Group Relative Policy Optimization (GRPO) cho giai đoạn Online RL. Đây là giai đoạn tận dụng khả năng của mô hình phần thưởng (reward model) để nhận diện các sắc thái trong chất lượng phản hồi.

- Mục tiêu: giúp mô hình tạo ra các câu trả lời chính xác, mạch lạc và cấu trúc tốt, đồng thời đảm bảo an toàn và dễ đọc. Các tiêu chí cụ thể bao gồm: tính trung thực, tính hữu ích, sự ngắn gọn, tính liên quan, tính vô hại và không có định kiến (bias).
- Cơ chế hoạt động của GRPO:
    - Với mỗi câu query, mô hình sẽ lấy mẫu 8 phản hồi khác nhau.
    - Hệ thống ưu tiên các phản hồi có điểm số cao hơn trung bình để tối ưu hiệu quả học tập.
- Dữ liệu huấn luyện: gồm dữ liệu nguồn công khai và các tập lệnh nội bộ có độ phức tạp cao.

Một điểm đáng chú ý là đối với các phiên bản xử lý ngữ cảnh dài (như Qwen2.5-Turbo), quá trình RL (bao gồm cả GRPO) chỉ được thực hiện trên các hướng dẫn ngắn (short instruction) (dưới 32.768 token).

- Lý do là vì huấn luyện RL trên ngữ cảnh dài rất tốn kém về tài nguyên tính toán và hiện đang thiếu các mô hình phần thưởng có khả năng đánh giá chính xác các tác vụ dài.
- Dù vậy, nhóm nghiên cứu phát hiện ra rằng việc áp dụng RL trên các hướng dẫn ngắn vẫn giúp **tăng cường đáng kể khả năng căn chỉnh của mô hình** ngay cả trong các tác vụ có ngữ cảnh dài.

6. Đánh giá hiệu suất

- Mô hình flagship Qwen 2.5-72B-Instruct đạt hiệu suất vượt trội so với nhiều mô hình mở và đóng, cho kết quả tương đương Llama-3-405B-Instruct mặc dù kích thước nhỏ hơn gấp 5 6 lần.
- Trong các nhiệm vụ chuyên biệt như toán học và lập trình, Qwen 2.5 cho thấy sự cải thiện vượt bậc so với Qwen 2.
- Qwen 2.5-Turbo đạt độ chính xác 100% cho tác vụ passkey retrieval ở độ dài 1 triệu token.

Bảng so sánh hiệu suất tổng quát (Instruct Models)

Chỉ số Benchmark	Llama-3.1-70B	Llama-3.1-405B	Qwen2.5-72B-Instruct
MMLU-redux	83.0	86.2	**86.8**
Arena-Hard	55.7	69.3	**81.2**
MT-Bench	8.79	9.08	**9.35**

MMLU-Pro	66.4	73.3	71.1
LiveBench 0831	46.6	53.2	52.3

Bảng so sánh năng lực Coding (Mô hình tầm trung và Flagship)

Mô hình	HumanEval	MBPP	LiveCodeBench
Llama-3.1-8B	72.6	69.6	8.3
Gemma2-9B	68.9	74.9	18.9
Qwen2.5-7B-Instruct	**84.8**	**79.2**	**28.7**
Qwen2.5-72B-Instruct	**86.6**	**88.2**	**55.5**
Llama-3.1-405B	89.0	84.5	41.6

→ Qwen 2.5 có kết quả tương đương hoặc vượt qua cả Llama 405B dù số tham số ít hơn 5 6 lần ở 1 vài tác vụ.

4. Qwen3 Đột phá kiến trúc

Dòng mô hình Qwen3, được phát triển và công bố bởi Alibaba Cloud vào tháng 4 năm 2025, đánh dấu một bước phát triển quan trọng trong thiết kế

mô hình ngôn ngữ lớn. Không chỉ đơn thuần kế thừa từ thế hệ Qwen2.5, Qwen3 giới thiệu một loạt cải tiến kiến trúc nhằm giải quyết những hạn chế cố hữu của Transformer truyền thống khi huấn luyện ở quy mô hàng trăm tỷ tham số.

4.1. Đột phá Kiến trúc Cốt lõi: Dense và MoE Tiêu chuẩn

Các mô hình Dense trong Qwen3 duy trì tính tương thích với những framework triển khai phổ biến như vLLM, SGLang, llama.cpp và Ollama. Đồng thời, chúng giữ lại nhiều thành phần quan trọng từ Qwen2.5 nhằm đảm bảo tính ổn định trong huấn luyện và hiệu suất suy luận. Các thành phần kế thừa tiêu biểu bao gồm:

- **Grouped Query Attention (GQA)**: tối ưu chia sẻ KV-cache giữa các query heads, giảm băng thông bộ nhớ và tăng tốc độ decoding.
- **SwiGLU activation**: tăng cường phi tuyến tính và khả năng biểu diễn.
- **Rotary Positional Embeddings (RoPE)**: mở rộng khả năng mô hình hóa chuỗi dài.
- **RMSNorm kết hợp pre-normalization**: đảm bảo ổn định gradient trong quá trình huấn luyện sâu.

Đáng chú ý, mô hình Qwen3-235B-A22B sử dụng cấu hình **64 query heads** và chỉ **4 KV heads**, thể hiện chiến lược tối ưu hóa chi phí tính toán nhưng vẫn đảm bảo hiệu quả suy luận.

Phân loại mô hình	Biến thể	Hiệu suất và Ứng dụng
Dense Model	0.6B, 1.7B, 4B, 8B,	Thay thế hoàn toàn các

	14B, 32B	mô hình Qwen2.5 có kích thước lớn hơn. Phù hợp cho việc triển khai cục bộ (local deployment).
MoE (lightweight)	30B, 3B (A3B)	Vượt trội hơn kiến trúc chuyên suy luận QwQ-32B với chi phí tính toán chỉ bằng 1/10.
MoE (flagship)	235B, 22B (A22B)	Đạt hiệu suất ngang hàng với các mô hình độc quyền hàng đầu như o1, o3-mini, Gemini 2.5 Pro.

Ngoài ra, kiến trúc Dense của Qwen3 tích hợp hai thay đổi vi mô quan trọng:

- **Loại bỏ hoàn toàn QKV-bias**

  Việc loại bỏ bias trong các phép chiếu Q, K và V giúp giảm độ phức tạp tính toán và giảm khả năng xuất hiện hiện tượng overfitting khi huấn luyện ở quy mô lớn.

- **Tích hợp QK-Norm vào Attention Mechanism**

  Trong kiến trúc Transformer truyền thống, giá trị dot-product giữa Query và Key có xu hướng tăng đột biến theo chiều sâu, gây mất ổn định. QK-Norm chuẩn hóa độc lập Q và K trước khi tính dot-product, từ đó giữ phân phối attention ổn định, hỗ trợ gradient flow và duy trì tính ổn định trong các mô hình hàng trăm tỷ tham số.

Bên cạnh các mô hình Dense, Qwen3 phát triển nhánh mô hình **Standard MoE**, hướng đến tối ưu hóa khả năng chuyên biệt hóa (specialization) của từng expert. Kiến trúc MoE của Qwen3 được đặc trưng bởi:

- **128 experts**, nhưng router chỉ **kích hoạt 8 experts cho mỗi token**, đảm bảo hiệu quả tính toán.
- **Không sử dụng shared experts**, khác với nhiều thiết kế MoE phổ biến, qua đó buộc các expert học các chức năng chuyên biệt sâu hơn.
- **Global-batch Load Balancing Loss**: được thêm vào để ngăn tình trạng *routing collapse*, đảm bảo phân phối token đồng đều qua các experts. Cơ chế này tự động thêm penalty nếu router dồn quá nhiều token vào một nhóm expert cụ thể.

4.2. Cơ chế Hybrid Thinking và Thinking Budget

Một trong những đổi mới có tính nền tảng của dòng Qwen3 là cách **hiện thực hóa Chain-of-Thought (CoT)** như một **đặc tính kiến trúc tường minh**, thay vì dựa vào prompt ẩn như "hãy suy nghĩ từng bước". Điều này đưa năng lực lập luận vào đúng vị trí của nó, *một mô-đun nội bộ của mô hình*, qua đó tăng tính minh bạch và kiểm soát.

Qwen3 hợp nhất hai chế độ vận hành vào cùng một khung thống nhất:

(1) Thinking Mode

- Được thiết kế cho các tác vụ đòi hỏi tư duy sâu như lập luận logic, giải tích, hoặc lập trình.

- Mô hình tiến hành lập luận nội bộ (internal reasoning), tự kiểm tra các giả thuyết và sửa lỗi trước khi xuất đầu ra.
- Mọi nội dung suy luận đều được bao gói trong cặp thẻ **<think> ... </think>**, mang lại khả năng quan sát trực tiếp và minh bạch, đặc biệt hữu ích trong hệ thống Agentic AI và tool-calling.

(2) Non-Thinking Mode

- Tối ưu độ trễ thấp, phục vụ tác vụ đối thoại hằng ngày, tìm kiếm thông tin, tóm tắt.
- Bỏ qua pipeline reasoning để đạt tốc độ phản hồi cực nhanh.

Cơ chế chuyển đổi giữa hai trạng thái có thể thực hiện qua:

- **system prompts**: /think, /no_think
- **API flags**: enable_thinking=False
- Hoặc tự động dựa trên loại nhiệm vụ.

Điều này cho phép ứng dụng cân bằng giữa **chất lượng lập luận** và **hiệu năng thời gian thực**.

Để tránh hiện tượng *overthinking*, mô hình suy luận nội bộ quá dài, gây lãng phí tài nguyên hoặc thậm chí timeout, Qwen3 triển khai tham số **thinking_budget** trên vLLM và Vertex AI. Tác dụng của Thinking Budget:

- **Tỷ lệ thuận chất lượng**: ngân sách suy luận dài hơn giúp mô hình khám phá sâu các giả thuyết khó, đặc biệt trong toán và lập trình.
- **Automatic Halting**: khi chạm giới hạn token, hệ thống tự động chèn thẻ </think> để buộc dừng quá trình lập luận.

- **Tinh chỉnh chi phí – latency – chất lượng**: nhà vận hành tùy chỉnh ngân sách để phù hợp tài nguyên, kỳ vọng chất lượng và mức trễ cho phép.

Nhờ cơ chế này, Qwen3 duy trì tính hiệu quả ngay cả trong các môi trường yêu cầu throughput rất cao.

4.3. Dữ liệu Siêu quy mô và Chiến lược Pre-training

Qwen3 được train trên 36 nghìn tỷ tokens (gấp đôi bản 2.5), bao phủ 119 ngôn ngữ toàn cầu. Quy mô này đẩy mạnh khả năng cross-lingual understanding, giảm sự phụ thuộc vào tiếng Anh, vốn là điểm lệch phổ biến của nhiều LLM hiện nay. Để thu thập nguồn dữ liệu này, đội ngũ thiết lập Synthetic Data Flywheel:

- **Found Data Extraction:** Dùng Vision-Language Model (Qwen2.5-VL) chạy OCR để trích xuất text/hình/toán từ PDF phi cấu trúc, sau đó dùng LLM để tinh chỉnh.
- **Synthetic Data Generation:** Huy động Qwen2.5-Math và Qwen2.5-Coder tạo ra nguồn cung cấp dữ liệu logic chuyên biệt (sách giáo khoa ảo, mã nguồn).

Quá trình Pre-training được chia thành 3 giai đoạn:

Giai đoạn	**Khối lượng dữ liệu**	**Độ dài ngữ cảnh**	**Mục tiêu**

S1 – Nền tảng	>30 nghìn tỷ	4,096 tokens	Xây dựng từ vựng, kiến thức chung đa ngôn ngữ; chủ yếu từ web corpus.
S2 – Suy luận	~5 nghìn tỷ	4,096 tokens	Tăng cường ngữ liệu STEM, toán và lập trình nhằm kích thích khả năng suy diễn quy nạp.
S3 – Ngữ cảnh dài	Hàng trăm tỷ	32,768 tokens	Huấn luyện chuyên biệt cho long-context; kết hợp thuật toán YaRN để ngoại suy lên 131K – 1M tokens.

4.4. Post-training và Kỹ thuật Chưng cất (Distillation)

Giai đoạn post-training của Qwen3 sử dụng một pipeline RL tinh vi gồm **4 bước**, nhằm kết hợp khả năng lập luận với kỹ năng đối thoại tự nhiên.

- Long-CoT Cold Start: Áp dụng SFT (Supervised Fine-Tuning) với phương pháp rejection sampling để lọc các quỹ đạo lập luận đúng.
- Reasoning RL: Dùng Rule-based reward và hệ thống verifiers cứng để truyền tín hiệu thưởng ngược lại mạng lưới.
- Thinking Mode Fusion: Trộn dữ liệu phản hồi nhanh (văn bản sáng tạo, nhập vai) để mô hình tự học cách phân biệt khi nào cần <think>, tránh thoái hóa kỹ năng giao tiếp.

- General RL: Triển khai hệ thống phần thưởng trên 20 tác vụ để giảm ảo giác (hallucination) và cải thiện gọi tool calls.

Strong-to-Weak Distillation: để giảm 90% chi phí train các mô hình nhỏ bằng cách dùng Flagship models (235B/32B) làm Teacher dẫn dắt Student models thông qua 2 pha:

- Off-policy Distillation:
  - Mô hình học sinh sao chép trực tiếp output "chuẩn" từ giáo viên.
  - Học cách lập luận (reasoning trajectories) và chuẩn hóa cú pháp thẻ **<think>**.
  - Truyền đạt kiến thức mang tính quy tắc.
- On-policy Distillation
  - Chuyển giao *dark knowledge* từ giáo viên sang học sinh.
  - Ép phân phối xác suất (logits) của học sinh khớp với giáo viên bằng tối ưu KL Divergence.
  - Học sinh không chỉ học câu trả lời đúng, mà còn học mức độ "không chắc chắn" (uncertainty) — một yếu tố quan trọng để giảm hallucination.

4.5. Đánh giá Hiệu năng, Scaling Laws và Năng lực Tiếng Việt

Nghịch lý Kích thước: Scaling Equivalence Sự kết hợp giữa dữ liệu chất lượng cao và fine-tuning giúp các mô hình Qwen3 Dense base phá vỡ định luật scaling laws:

- Qwen3-1.7B-Base tương đương Qwen2.5-3B-Base.
- Qwen3-4B-Base tương đương Qwen2.5-7B-Base.

- 8B và 14B thay thế hoàn toàn dòng 14B và 32B cũ.
- Qwen3-32B-Base có hiệu năng tổng hợp ngang hàng với cỗ máy 72B-Base khổng lồ của thế hệ trước. Tỷ lệ performance-to-parameter vượt trội cực rõ rệt ở mảng STEM và coding. Mô hình MoE 30B-A3B đánh bại QwQ-32B trên các thang đo lập luận dù tham số kích hoạt chỉ bằng 1/10.

Sức mạnh Flagship Mô hình cờ đầu Qwen3-235B-A22B cạnh tranh trực tiếp với o1 và Gemini 2.5 Pro. Mô hình ghi nhận 85.7 AIME'24 và điểm chuẩn MMLU trung bình tĩnh là 82.9 (có thể đạt trần 95.5 ở Sparse Attention). Mảng vision, nhánh VL-Embedding-8B vươn lên top 1 toàn cầu trên MMEB-V2 (77.8).

Việc mở rộng ngân hàng dữ liệu lên 119 ngôn ngữ mang lại hiệu quả cực lớn cho Tiếng Việt trên thang đo VMLU.

- Qwen3 khắc phục lỗi ngữ nghĩa, mất diacritics (dấu tiếng Việt) và giải quyết hiện tượng lai tạp cấu trúc ngữ pháp Anh (semantic contamination) thường thấy ở các model cũ.
- Pha đào tạo RL (Instruction Following) đóng vai trò như màng lọc nhiễu ngôn ngữ, giúp 235B-A22B thiết lập điểm chuẩn vượt trội (Zero-shot).
- Điều này đưa Qwen3 trở thành nền tảng lõi cho các dịch vụ chatbot AI và hệ thống dịch thuật Qwen-MT tại thị trường Việt Nam giai đoạn 2025-2026.

5. Sự cải thiện của Qwen-VL qua từng phiên bản

5.1. Kiến trúc Qwen-VL Gốc và các Bottlenecks

5.1.1 Kiến trúc Cơ sở và Giới hạn Độ phân giải Cố định

Phiên bản Qwen-VL đầu tiên được xây dựng theo mô hình kiến trúc điển hình của các MLLM giai đoạn sơ khai. Hệ thống gồm ba thành phần: **Vision Encoder** (thường được triển khai bằng **Vision Transformer – ViT**) để trích xuất đặc trưng thị giác, **LLM** đóng vai trò bộ máy suy luận ngữ nghĩa, và một **Adapter** để ánh xạ không gian biểu diễn giữa hai miền thị giác–ngôn ngữ.

Trong pipeline này, hình ảnh được chia thành các *patches* và xử lý qua các lớp của ViT để tạo ra tập hợp các *visual tokens*. Tuy nhiên, toàn bộ kiến trúc chịu ràng buộc bởi một hạn chế nghiêm trọng: **Predetermined (Fixed) Resolution**. Mọi ảnh đầu vào—bất kể kích thước hoặc *aspect ratio*—đều phải bị ép về một độ phân giải hình vuông duy nhất (224×224, 336×336, …) thông qua phép *resize* hoặc *center crop*.

Hệ quả của ràng buộc này là **biến dạng hình học** và **mất mát chi tiết cấp pixel**, đặc biệt nghiêm trọng đối với các tác vụ yêu cầu độ phân giải cao (high-fidelity), như OCR trên tài liệu dài hoặc phát hiện đối tượng nhỏ trong ảnh có trường nhìn rộng. Việc nội suy và nén hình học trước khi đưa vào ViT khiến mô hình mất quyền truy cập vào các chi tiết quan trọng ngay từ giai đoạn tiền xử lý, tạo ra giới hạn cứng về năng lực nhận thức thị giác.

Một cách hình tượng, kiến trúc này tương tự như việc buộc mọi bức ảnh, dù là ảnh panorama ngang dài hay ảnh tài liệu dọc, phải "nhét" vào một khuôn

vuông duy nhất. Quá trình ép khuôn này làm biến dạng tỷ lệ đối tượng và xóa nhòa thông tin nhỏ, đánh lừa mô hình về cấu trúc thực sự của cảnh quan.

5.1.2. Nút thắt Thông tin tại Khối Resampler

Bên cạnh rào cản độ phân giải, một thách thức cơ bản khác đến từ **Adapter module**, cụ thể là **Resampler** (hay **Q-Former** trong các thiết kế tương đương). Do **LLM** bị giới hạn bởi **context window** và chi phí attention tăng theo $O(N^2)$, số lượng *visual tokens* từ ViT (thường hàng nghìn tokens) không thể đưa trực tiếp vào LLM.

Để khắc phục, Resampler sử dụng một tập *latent queries* để **nén** không gian thị giác xuống một số lượng rất nhỏ *projected tokens* (ví dụ: 64–256 tokens). Tuy nhiên, quá trình nén này có bản chất **lossy**—nó không bảo toàn dữ liệu như hình chiếu tuyến tính, mà chọn lọc thông tin theo phân bố dữ liệu huấn luyện. Điều này dẫn đến **mất mát chi tiết vi mô không thể phục hồi**, làm suy giảm khả năng suy luận ở những nhiệm vụ đòi hỏi phân tích đa cấp độ hoặc nhận diện chi tiết nhỏ. Có thể hình dung ViT thu thập "hồ sơ" gồm hàng nghìn chi tiết của cảnh quan, nhưng Resampler buộc phải rút gọn toàn bộ xuống một bản tóm tắt khoảng vài trăm từ trước khi gửi đến LLM. Dù được tối ưu tốt, quá trình này vẫn loại bỏ nhiều bằng chứng quan trọng, những "vết xước nhỏ" hoặc "chữ mờ trên biên lai", khiến LLM không thể thực hiện suy luận tinh vi dựa trên ngữ cảnh thị giác đầy đủ.

5.2. Kiến trúc Qwen2-VL: Breaking the Mold

Sự ra đời của **Qwen2-VL** đánh dấu bước chuyển mình quan trọng trong thiết kế MLLM khi không chỉ đơn thuần là cải tiến mô hình, mà là sự thay đổi

paradigm trong cách mạng nơ-ron đa phương thức mã hóa không gian, hình học và cấu trúc thông tin. Phiên bản này giới thiệu hai đổi mới cốt lõi: **Naive Dynamic Resolution** và **M-RoPE (Multimodal Rotary Position Embedding)**. Hai thành phần này trực tiếp loại bỏ các giới hạn cơ bản của thế hệ Qwen-VL đầu tiên.

5.2.1. Naive Dynamic Resolution: Khả năng xử lý hình ảnh với độ phân giải động

Một trong những bài toán khó nhất của thị giác trong MLLM là khả năng thích ứng với sự đa dạng cao của hình ảnh thực tế. **Qwen2-VL** giải quyết triệt để rào cản "fixed resolution" bằng cách triển khai **Naive Dynamic Resolution**, cho phép mô hình:

- Tiếp nhận hình ảnh ở bất kỳ *aspect ratio* tự nhiên nào,
- Tạo ra số lượng *visual tokens* thay đổi theo kích thước thật,
- Duy trì tính nhất quán của thông tin không gian mà không cần ép khuôn.

Thay vì resize hình ảnh về một kích thước cố định, hệ thống vận hành dựa trên lưới **block-based**:

1. Hình ảnh được điều chỉnh nhẹ để chia hết cho kích thước patch.
2. Hình ảnh được chia thành **patches** kích thước cơ bản 14×14.
3. Adapter mới của Qwen2-VL gộp **khối 2×2** patches thành **một token duy nhất**, tương đương một vùng 28×28 pixel.
4. Số lượng token được tính bằng công thức:

$$Image\ Tokens = \frac{h_{bar}\ X\ \omega_{bar}}{784} + 2$$

với 784=28×28 là số pixel đại diện cho một token.

Các ràng buộc về bộ nhớ được kiểm soát thông qua **min_pixels** và **max_pixels**, đảm bảo mô hình không vượt quá giới hạn VRAM.

Lợi ích của kiến trúc động là khác với thế hệ cũ, vốn lãng phí token vào vùng padding và gây biến dạng hình học, Dynamic Resolution loại bỏ hoàn toàn padding, bảo toàn tỷ lệ vật thể, phân bổ số lượng token tỷ lệ thuận với mật độ thông tin và mô phỏng gần hơn cơ chế nhận thức thị giác sinh học.

5.2.2. Loại bỏ Resampler: Kiến trúc MLP-based Merger tối giản và hiệu quả

Đi kèm với độ phân giải động, Qwen2-VL đã thực hiện một cuộc phẫu thuật mạnh tay khi loại bỏ hoàn toàn cơ chế Resampler/Q-Former phức tạp của thế hệ trước. Để thay thế, các kỹ sư kiến trúc tận dụng một Mạng nơ-ron truyền thẳng (MLP - Multilayer Perceptron) đơn giản nhưng mang lại hiệu suất cao đến kinh ngạc. Mạng gộp thị giác-ngôn ngữ (Vision-Language Merger) này có nhiệm vụ thu thập và gom nhóm các cụm đặc trưng không gian liền kề (theo khối 2 X 2 patch) từ mạng ViT, nối chúng lại với nhau thành một khối dữ liệu duy nhất, và sau đó chiếu khối này sang không gian chiều của các vector nhúng văn bản của LLM thông qua một mạng MLP gồm 2 lớp tuyến tính.

Lý do cho sự cải tiến này đến từ việc phân tích tổn thất huấn luyện. Resampler tuy nén dữ liệu rất tốt, nhưng việc ép mạng nơ-ron phải học các

ánh xạ (mappings) ẩn giấu thông qua cơ chế chú ý chéo (cross-attention) là một quá trình cực kỳ khó khăn, tốn kém tài nguyên tính toán và thường dẫn đến tình trạng hội tụ chậm trong huấn luyện. Thêm vào đó, Resampler phá vỡ tính liên tục không gian của ảnh. Cấu trúc gộp 2 X 2 qua MLP, ngược lại, làm giảm 4 lần chiều dài chuỗi token (sequence length) một cách tuyến tính, minh bạch và xác định. Nó thực hiện việc nén mà vẫn giữ lại cấu trúc không gian nguyên bản thay vì nội suy dự đoán. Quyết định loại bỏ sự phức tạp không cần thiết để đổi lấy cấu trúc tuyến tính vững chắc này đã giúp mô hình lưu giữ các chi tiết vi mô cho các bức ảnh phân giải cao, dẫn đến sự cải thiện vượt bậc trong đánh giá điểm chuẩn so với Qwen-VL cũ.

5.2.3. Multimodal Rotary Position Embedding (M-RoPE): Tọa độ Hình học 1D–2D–3D

Nếu Dynamic Resolution giải quyết vấn đề về *kích thước không gian*, thì **M-RoPE** giải quyết vấn đề về *bản chất hình học*. Đây là cải tiến toán học quan trọng nhất trong Qwen2-VL.

Quay lại với giới hạn của RoPE 1D truyền thống, RoPE gốc được thiết kế cho ngôn ngữ, một cấu trúc 1D, bằng cách áp dụng phép xoay trong không gian phức để mã hóa vị trí tương đối của token trong một chuỗi.

Nhưng khi áp dụng cho hình ảnh, điểm yếu sẽ bị lộ ra khi ảnh 2D bị flatten thành 1D, quan hệ không gian bị phá vỡ hoàn toàn khiến hai pixel xa nhau có thể trở thành “hàng xóm” trong chuỗi và mô hình mất khả năng suy luận hình học.

Giải pháp của alibaba là cải tiến thành **M-RoPE (Multimodal Rotary Positional Embedding)**. M-RoPE phân tách phép xoay vị trí thành ba thành phần độc lập:

- **Temporal (1D)**: trục thời gian cho video
- **Spatial Height (1D)**: trục dọc của ảnh
- **Spatial Width (1D)**: trục ngang của ảnh

Các trục này vận hành trên các *frequency bands* khác nhau của vector embedding, giúp LLM đồng thời tiếp nhận tín hiệu vị trí 1D (text) + 2D (image) + 3D (video) mà không bị nhiễu chéo.

Từ đó LLM có khả năng "nhìn" bằng tọa độ thật khi M-RoPE cung cấp cho mô hình khả năng nhận thức đúng về khoảng cách Euclidean trong ảnh, duy trì quan hệ hình học đa chiều, có sự ổn định khi kết hợp các modality khác nhau và cuối cùng có khả năng mở rộng tự nhiên sang video và sequence dài

5.3. Kiến trúc Qwen2.5-VL: Hyper-speed processing

Dựa trên nền tảng hình học đã được củng cố từ các thế hệ trước, dòng mô hình **Qwen2.5-VL** xuất hiện như một bước nhảy vọt, định vị mình không chỉ là một công cụ phân tích hình ảnh mà là một **Visual Agent** toàn diện, có khả năng quan sát, suy luận, và tương tác tự chủ với môi trường số (GUI, trình duyệt, thiết bị di động). Phiên bản này giới thiệu những cải tiến kiến trúc cốt lõi ở **Vision Transformer (ViT)**, hệ tọa độ, cùng một paradigm huấn luyện hoàn toàn mới xoay quanh **Reinforcement Fine-Tuning (RFT)**.

5.3.1. Cuộc cách mạng Attention: Window Attention trong ViT

Một trong những rào cản lớn nhất khi mở rộng quy mô mô hình đa phương thức chính là chi phí tính toán khổng lồ của **Global Self-Attention** trong ViT. Độ phức tạp $O(N^2)$ khiến việc xử lý ảnh độ phân giải động (dynamic resolution) trở thành một nút thắt cổ chai nghiêm trọng, đặc biệt khi một ảnh có thể sinh ra *vài chục nghìn* visual tokens.

Để khắc phục triệt để, Qwen2.5-VL tái thiết kế ViT bằng cách áp dụng triệt để **Window Attention**. Thay vì yêu cầu mỗi patch trong ảnh phải "nhìn" toàn bộ các patch khác, mô hình chia ảnh thành các **local windows** (tối đa kích thước 8×8 patch ≈ 112×112 px) và thực hiện attention *chỉ bên trong cửa sổ đó*. Nhờ cơ chế locality này, FLOPs được cắt giảm **5–10 lần** trên các ảnh lớn, đồng thời loại bỏ hiện tượng load imbalance giữa ViT và LLM.

Tuy nhiên, nếu chỉ dùng Window Attention, mô hình sẽ trở nên "cận thị". Vì vậy Qwen2.5-VL vẫn giữ lại **4 lớp Global Attention** ở các vị trí chiến lược để duy trì **global context routing**. Sự kết hợp này đảm bảo hai yếu tố:

- Tốc độ → tối ưu hóa inference và training
- Chất lượng → bảo toàn thông tin toàn cục khi cần reasoning cấp cao

Kiến trúc ViT cũng được chuẩn hóa về đường dẫn tính toán qua **RMSNorm** và **SwiGLU** để đồng bộ với LLM backbone, giúp giảm phân kỳ khi huấn luyện.

5.3.2. Absolute Coordinate System và Dynamic FPS Training

Để mô hình hoạt động như một agent thực thụ, Qwen2.5-VL cần hiểu **hình học thực tế**, thay vì các tỷ lệ tương đối bị chuẩn hóa.

Absolute Coordinates

Các mô hình cũ thường chuẩn hóa bounding box về một quy mô tương đối. Điều này khiến mô hình mất cảm nhận về kích thước thật của các vật thể, gây lỗi khi điều khiển GUI: bấm lệch nút, click sai vị trí, thao tác trượt.

Qwen2.5-VL thay thế hoàn toàn bằng Absolute Size Scale (toạ độ tuyệt đối theo pixel) khi input/output giữ nguyên đơn vị pixel và không có bước scale hoặc normalization

Điều này tăng mạnh độ chính xác cho các tác vụ:

- UI automation
- robot process automation (RPA)
- grounding chính xác đến từng pixel
- sinh JSON structured output ổn định hơn

Dynamic FPS + Absolute Time Encoding

Trong dữ liệu video, Qwen2.5-VL thay thế việc lấy mẫu fixed-FPS bằng **Dynamic FPS training**, giúp mô hình chịu được quá trình thay đổi tốc độ cũng như drop frames và tốc độ video bất quy tắc trong môi trường thật

Đặc biệt quan trọng hơn, mô hình sử dụng Absolute Time Encoding (ATE) giúp M-RoPE ID trong chiều thời gian không còn là các chỉ số tăng đều mà được **gắn với timestamp thật** của video. Điều này cho phép các event localization chính xác đến từng giây, nhận thức timeline liên tục và phi tuyến tính, trả lời được câu hỏi dạng: "sự kiện xảy ra ở 14:32.45 giây?"

5.3.3. Tinh chỉnh tăng cường đặc vụ: Visual-RFT và Visual-ARFT

Một Visual Agent không chỉ cần kiến trúc tốt mà còn cần "hệ điều hành nhận thức" tối ưu. Qwen2.5-VL lần đầu triển khai **Reinforcement Fine-Tuning (RFT)** cho thị giác, với hai biến thể chính:

- **Visual-RFT** nhằm tối ưu reasoning
- **Visual-ARFT** nhằm tối ưu hành vi agentic (GUI control, coding, manipulation)

Vì sao SFT thất bại? SFT dạy mô hình *bắt chước* (behavior cloning) bằng cách đưa input và lời giải mẫu xong yêu cầu mô hình lặp lại. Khi môi trường đổi khác hoặc có sự nhiễu, mô hình dễ "vỡ chuỗi", vì nó không được học cách **thử, sai và sửa**.

Qwen2.5-VL sử dụng **GRPO (Group Relative Policy Optimization)** cùng hệ thống **rule-based verifiable rewards**. Giúp Mô hình sinh ra **nhiều** lời giải song song. Chỉ lời giải nào *thỏa reward* mới nhận điểm:

- JSON: **schema-valid** mới được chấp nhận
- Grounding: phải vượt ngưỡng **IoU**

- Coding: đoạn code phải **chạy thành công**
- Agentic action: click đúng pixel mục tiêu

Nhờ vậy, mô hình học cách **tự khám phá**, không sao chép.

5.4. Kiến trúc Qwen3-VL: Định hình lại Không gian Đa phương thức

Qwen3-VL được thiết kế lại từ nền tảng để giải quyết ba điểm nghẽn cốt lõi của các Vision-Language Models (VLMs) truyền thống:
(1) mất chi tiết do nén ảnh,
(2) nghẽn băng thông khi truyền visual tokens vào LLM,
(3) suy luận không ổn định trên chuỗi dài và video kéo dài nhiều giờ.

Kiến trúc mới tổ chức hệ thống thành ba mô-đun chính: **Vision Encoder**, **Vision-Language Merger**, và **LLM Backbone Qwen3**, được kết nối thông qua hệ tọa độ động và các cơ chế tiêm đặc trưng đa tầng. Nhờ đó, Qwen3-VL không chỉ xử lý ảnh và video vượt trội mà còn mở rộng sang grounding 3D và logic liên khung hình phức tạp.

5.4.1. Vision Encoder & Dynamic Resolution

Thay vì sử dụng CLIP như thế hệ VLM cũ, Qwen3-VL chuyển sang **SigLIP-2** (SigLIP2-SO-400M cho model lớn và SigLIP2-Large cho bản 2B/4B). Sự thay đổi này cho phép mô hình xử lý hình ảnh theo **độ phân giải gốc (native resolution)**.

Điểm mấu chốt của cơ chế Dynamic Resolution:

- Không còn resize/pad cứng thành hình vuông — vốn dễ làm méo hình, mất bối cảnh.
- Mô hình dùng **2D-RoPE** kết hợp **interpolated absolute positional embedding** để "kéo dãn" hệ tọa độ vị trí theo đúng độ phân giải thực tế.
- Qwen3-VL giữ nguyên bố cục không gian, đặc biệt quan trọng cho OCR phức tạp, bản vẽ, hoặc ảnh có chi tiết nhỏ.

Cách mã hóa linh hoạt này đặt nền móng cho các mô-đun sau, đảm bảo rằng mọi thông tin thị giác tiếp tục được bảo toàn trong suốt pipeline.

5.4.2. Vision-Language Merger & Cơ chế nén C2T

Sau khi nhận dữ liệu từ Vision Encoder, Merger đóng vai trò "cổng kết nối" tối ưu hóa luồng thị giác trước khi đưa vào LLM.
Qwen3-VL áp dụng cơ chế **C2T (Compress 2×2)**:

- Mỗi cụm **2×2 visual patches** được gộp lại thông qua **MLP hai tầng** để tạo thành **một visual token duy nhất**.
- Độ dài chuỗi token thị giác giảm còn **25%**, nhưng thông tin vẫn được bảo toàn ở mức cần thiết.

C2T giúp giảm áp lực công suất trên LLM trong các tác vụ đa ảnh hoặc ảnh độ phân giải cao. Đồng thời giảm chi phí VRAM khi fine-tuning, cho phép mở rộng context window rất lớn. Từ đó giúp mô hình xử lý nhiều hình ảnh trong cùng một truy vấn mà vẫn đảm bảo tính gắn kết thông tin.

5.4.3. Đột phá Cơ chế Bơm Đặc trưng: DeepStack

Các VLM cũ thường gặp lỗi OCR routing bottleneck (mất chi tiết nhỏ) do Vision Encoder chỉ nén toàn bộ ảnh và đẩy một cục visual tokens vào đầu LLM. DeepStack thay đổi luồng dữ liệu này bằng cách "tiêm" thông tin trực tiếp vào quá trình suy luận nội bộ:

- Multi-level Extraction: DeepStack trích xuất các features từ nhiều cấp độ (intermediate layers) của Vision Transformer. Nó lấy cả thông tin thô (cạnh, màu sắc ở lớp thấp) lẫn ngữ nghĩa (nhận diện khối ở lớp cao).
- Direct Injection: Các luồng dữ liệu này đi qua Merger chuyên dụng, được chiếu (projected) sang hidden dimension của LLM và cộng trực tiếp vào hidden states của 3 lớp đầu tiên thông qua residual connections.
- Ứng dụng khi Fine-tune: Cơ chế này đặc biệt giá trị khi xây dựng các hệ thống Visual Question Answering (VQA) chuyên sâu. Ví dụ, khi phân tích các tài liệu học thuật phức tạp hoặc nghệ thuật Phật giáo, DeepStack giúp mô hình nhận diện xuất sắc các chi tiết vật lý cấp thấp như ấn triện mờ, kết cấu vật liệu, hay nét chữ cổ cực nhỏ mà không hề làm phình to Context Length.

5.4.4. 3D Grounding & Affordance Learning

Qwen3-VL chuyển đổi từ "nhìn mặt phẳng 2D" sang "hiểu khối không gian 3D", mở ra tiềm năng tích hợp cho Agentic AI trong môi trường thực tế.

- Virtual Camera Coordinate System: Hệ thống chuẩn hóa mọi dữ liệu (từ LiDAR, RGB-D) về một hệ tọa độ camera ảo thống nhất, giúp mô hình nắm bắt không gian độc lập với phần cứng.

- 9-DoF 3D Bounding Boxes: Khi cần trích xuất vị trí, thay vì trả về bounding box 2D thông thường, mô hình xuất ra cấu trúc JSON 9 bậc tự do (9-DoF) gồm: Tọa độ tâm 3D (x, y, z), kích thước (length, width, height), và góc quay không gian (roll, pitch, yaw).
- Affordance Learning: AI được học các nhãn tương tác (sittable, graspable) kết hợp tọa độ chuẩn hóa. Nó không chỉ biết vật thể ở đâu, mà còn ước lượng độ sâu và khả năng tương tác vật lý.

5.4.5. Multi-Image Understanding (Hiểu đa ảnh)

Với Interleaved Context (ngữ cảnh đan xen) lên tới 256.000 tokens, Qwen3-VL thiết lập luồng suy luận mạch lạc xuyên suốt chuỗi:

[Image 1] → [Text 1] → [Image 2] → [Text 2] → ...

- Book-scale Data: Mô hình được huấn luyện trên dữ liệu có cấu trúc giống sách và tài liệu dài, giúp duy trì mạch logic xuyên suốt bộ ảnh hoặc chuỗi trang.
- Cross-Image Reasoning: Hệ thống thực hiện ánh xạ tương đồng (Visual Correspondence), phát hiện khác biệt (Difference Detection) và suy luận đa bước (Multi-hop Reasoning).
- Ứng dụng khi Fine-tune: Khả năng xử lý logic nhân quả và sự thay đổi trạng thái giữa các khung hình (frames) này là nền tảng hoàn hảo để huấn luyện AI tự động hóa việc phân tích storyboards hoặc tự động khởi tạo Webtoon từ các kịch bản (scripts) văn bản, đảm bảo tính nhất quán tuyệt đối của nhân vật và bối cảnh qua từng phân cảnh.

5.4.6. Xử lý Video Siêu dài với Interleaved MRoPE và Explicit Textual Timestamps

Để xử lý video kéo dài hàng giờ với FPS thay đổi, Qwen3-VL áp dụng hai cải tiến lớn:

- Interleaved MRoPE – Phối trộn tần số thời gian–không gian
  - Thay vì mã hóa vị trí theo từng khối tách biệt [t | h | w] như trước, Qwen3-VL dùng cấu trúc trộn: [t, h, w, t, h, w, ...]
  - Kết quả là trục thời gian *(t)* có đủ tần số thấp → nhớ dài hạn. trục không gian *(h, w)* có đủ tần số cao → giữ chi tiết chuyển động. Từ đó, mô hình hiểu rõ cả **motion tức thời** và **logic dài hạn**.
- Explicit Textual Timestamps – Gắn mốc thời gian bằng từ ngữ
  - Thay thế toàn bộ absolute position IDs bằng **timestamp dạng text**, ví dụ:
    - <3.0 seconds>
    - <01:05:20>
  - Ưu điểm:
    - Không bị lỗi khi FPS thay đổi hoặc dữ liệu bị skip.
    - LLM hiểu thời gian như một khái niệm ngôn ngữ, giúp phân tích trình tự sự kiện chính xác hơn.
    - Phù hợp cho video dài, giám sát, documentary, camera an ninh.

6. Qwen3.5 Kỷ nguyên AI Agents

6.1. Tổng quan về Sự ra mắt của Qwen3.5

Vào ngày **15/02/2026**, đội ngũ Qwen đã chính thức công bố **Qwen3.5**, đánh dấu một bước chuyển mình từ các trợ lý AI thông thường sang các **Agent kỹ thuật số phổ quát**,. Mô hình đầu tiên được mở trọng số (open-weight) trong series này là **Qwen3.5-397B-A17B**, một mô hình ngôn ngữ-thị giác bản địa (native vision-language model). Mục tiêu của thế hệ này là tối ưu hóa năng suất cho các nhà phát triển và doanh nghiệp thông qua khả năng suy luận, lập trình và xử lý đa phương thức vượt trội.

6.2. Kiến trúc Hybrid Đột phá và Hiệu năng Tính toán

Qwen3.5 được xây dựng trên một kiến trúc lai (hybrid architecture) sáng tạo, giúp giải quyết bài toán cân bằng giữa quy mô tham số và chi phí vận hành:

- **Sự kết hợp Gated Delta Networks & MoE:** Mô hình kết hợp giữa **Linear Attention** (thông qua Gated Delta Networks) và cơ chế **Mixture-of-Experts (MoE)** thưa,.
- **Tham số kích hoạt thông minh:** Mặc dù tổng số lượng tham số lên tới **397 tỷ (397B)**, nhưng chỉ có **17 tỷ (17B) tham số được kích hoạt** trong mỗi lượt truyền xuôi (forward pass),.
- **Tốc độ giải mã (Throughput) cực nhanh:** Nhờ kiến trúc mới, tốc độ giải mã của Qwen3.5-397B-A17B nhanh hơn từ **8,6 lần đến 19 lần** so với mô hình Qwen3-Max tiền nhiệm (tùy theo độ dài ngữ cảnh 32k hay 256k).

- **Hiệu quả huấn luyện:** Qwen3.5-397B-A17B đạt được năng lực tương đương với các mô hình có quy mô hơn 1 nghìn tỷ tham số (như Qwen3-Max-Base) nhờ dữ liệu huấn luyện quy mô lớn và chất lượng cao.

6.3. Năng lực Đa phương thức và Agent Thị giác (Visual Agents)

Qwen3.5 không chỉ hiểu văn bản mà còn được thiết kế để "nhìn" và "hành động" trong thế giới kỹ thuật số:

Agent GUI (Giao diện đồ họa): Mô hình có khả năng tương tác tự động với điện thoại thông minh và máy tính. Nó có thể tuân theo chỉ dẫn bằng ngôn ngữ tự nhiên để thực hiện các tác vụ xuyên ứng dụng hoặc xử lý các luồng công việc văn phòng phức tạp trên máy tính.

Xử lý Video dài: Qwen3.5 hỗ trợ cửa sổ ngữ cảnh lên tới **1 triệu tokens** (phiên bản Plus), cho phép xử lý và hiểu nội dung của các video dài tới **2 giờ**,. Các ứng dụng thực tế bao gồm: chuyển bản vẽ tay thành mã nguồn frontend, tóm tắt video dài thành trang web hoặc phân tích logic từ video chơi game.

Trí tuệ Không gian (Spatial Intelligence): Bằng cách mô hình hóa mối quan hệ không gian ở cấp độ pixel, mô hình đạt độ chính xác cao trong việc đếm đối tượng, xác định vị trí tương đối và mô tả không gian. Điều này mở ra tiềm năng lớn cho các ứng dụng AI hiện hữu (embodied AI) như hiểu bối cảnh lái xe tự hành,.

6.4. Khả năng Đa ngôn ngữ và Dữ liệu Tiền huấn luyện

Hỗ trợ 201 ngôn ngữ: Phạm vi ngôn ngữ được mở rộng từ 119 lên **201 ngôn ngữ và phương ngữ**, đặc biệt ưu tiên các ngôn ngữ ít tài nguyên để thúc đẩy công bằng AI,. Danh sách bao gồm các ngôn ngữ phổ biến và các phương ngữ hiếm như tiếng Latgalian, tiếng Phạn, tiếng Khmer và tiếng Việt,.

Bộ từ vựng khổng lồ: Từ vựng được nâng cấp từ 150k lên **250.000 từ**, giúp tăng hiệu suất mã hóa/giải mã thêm 10-60% trên hầu hết các ngôn ngữ.

Quy mô dữ liệu: Mô hình được huấn luyện trên lượng mã báo (tokens) văn bản-thị giác lớn hơn nhiều so với Qwen3, với các tiêu chuẩn lọc dữ liệu cực kỳ khắt khe cho dữ liệu STEM và suy luận.

6.5. Đánh giá Hiệu năng (Benchmarks)

Qwen3.5-397B-A17B đã thể hiện sức mạnh vượt trội khi so sánh với các mô hình hàng đầu thế giới như GPT-5.2, Claude 4.5 Opus và Gemini-3 Pro:

Kiến thức & Suy luận: Đạt **87.8** điểm trên MMLU-Pro và **91.3** điểm trên AIME26 (Toán học),.

Lập trình & Agent: Đạt **76.4%** trên SWE-bench Verified, chứng minh khả năng giải quyết các vấn đề phần mềm thực tế. Trong các bài kiểm tra Agent tìm kiếm (Search Agent) như WideSearch, mô hình đạt tới **74.0** điểm.

Hiểu Thị giác (Vision): Dẫn đầu trong nhiều bài kiểm tra như Mathvista-mini (**90.3**) và RealWorldQA (**83.9**).

6.6. Hạ tầng và Khung học tăng cường (RL Framework)

Để đạt được hiệu suất này, Alibaba đã triển khai các cải tiến hạ tầng quan trọng:

Hạ tầng không đồng nhất (Heterogeneous Infrastructure): Cho phép tách biệt các chiến lược song song giữa thành phần thị giác và ngôn ngữ, đạt hiệu suất huấn luyện gần 100% so với chỉ huấn luyện văn bản.

Pipeline FP8: Áp dụng độ chính xác thấp (low precision) cho các phép toán MoE và GEMM, giúp giảm 50% bộ nhớ kích hoạt và tăng tốc độ hơn 10% mà vẫn giữ được sự ổn định.

Hệ thống RL không đồng bộ: Khung học tăng cường mới hỗ trợ đa phương thức và hội thoại nhiều lượt, giúp tăng tốc độ huấn luyện từ **3 đến 5 lần**.

6.7. Các chế độ tương tác và Truy cập

Người dùng có thể trải nghiệm Qwen3.5 thông qua ba chế độ linh hoạt trên Qwen Chat:

1. **Auto (Tự động):** Sử dụng tư duy thích ứng, tự động quyết định khi nào cần suy luận sâu hoặc sử dụng công cụ (như tìm kiếm web và trình thông dịch mã).
2. **Thinking (Suy luận):** Cho phép mô hình suy nghĩ sâu (Chain-of-Thought) để giải quyết các bài toán hóc búa,.
3. **Fast (Nhanh):** Phản hồi tức thì cho các yêu cầu đơn giản mà không tiêu tốn thêm tokens cho việc suy nghĩ.

Ngoài ra, phiên bản flagship **Qwen3.5-Plus** có sẵn trên Alibaba Cloud Model Studio, hỗ trợ cửa sổ ngữ cảnh **1 triệu tokens** mặc định và tích hợp các công cụ chuyên dụng cho doanh nghiệp,.

6.8 Kết luận

Qwen3.5 không chỉ đơn thuần là một bản nâng cấp về tham số, mà là một hệ thống AI có khả năng tự trị cao hơn, hoạt động mượt mà trong môi trường đa phương thức và sẵn sàng cho các nhiệm vụ thực thi phức tạp kéo dài nhiều ngày. Sự kết hợp giữa kiến trúc hiệu quả và khả năng suy luận bản địa đặt nền móng vững chắc cho tương lai của các Agent kỹ thuật số.

IV. Xây dựng mô hình và thiết lập thực nghiệm

1. Chuẩn bị và tiền xử lý dữ liệu

1.1. Giới thiệu về tập dữ liệu train MathVision

Tập Tập dữ liệu **MathVision (MATH-Vision)** được giới thiệu trong bài báo *"Measuring Multimodal Mathematical Reasoning with MATH-Vision Dataset" (2024)* nhằm đánh giá khả năng suy luận toán học đa phương thức của các mô hình AI hiện đại.

Khác với nhiều benchmark trước đây, MathVision được xây dựng từ các bài toán thực tế trong các kỳ thi toán học, bao gồm nhiều dạng hình ảnh như hình học, biểu đồ, bảng số liệu và sơ đồ. Điều này giúp dữ liệu mang tính thực tiễn cao và yêu cầu mô hình phải kết hợp cả hiểu hình ảnh và suy luận toán học.

Một đặc điểm quan trọng của MathVision là: - Không cung cấp chuỗi suy luận (reasoning chain) - Chỉ cung cấp câu hỏi, hình ảnh và đáp án cuối

Điều này tạo ra một hạn chế lớn trong việc huấn luyện các mô hình có khả năng suy luận từng bước.

Danh mục	**Chỉ số**	**Giá trị**
Tổng quan	Tổng số câu hỏi	3,040
	Câu hỏi trắc nghiệm	1,532 (50,4%)
	Câu hỏi tự luận	1,508 (49.06%)
	Câu hỏi Testmini	304 (10.0%)
Độ đa dạng	Số cấp độ	5
	Số môn học	16
	Cuộc thi nguồn	19
Tài nguyên	SL hình ảnh duy nhất	3,472
	SL đáp án duy nhất	370
Độ dài câu hỏi	Độ dài tối đa	280
	Độ dài tối thiểu	5
	Độ dài trung bình	42,3
Nguồn dữ liệu	Math Kangaroo	1,499
	UK (Grey, Pink, Junior, Senior)	535
	AMC (8, 10, 12) AIME	759

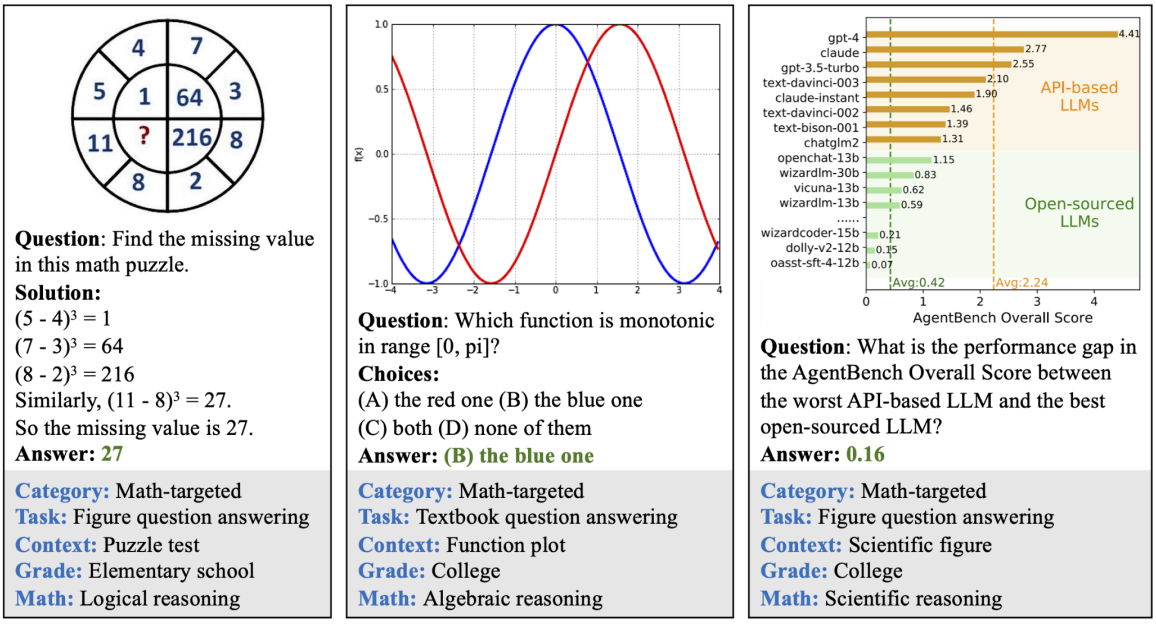
1.2 Giới thiệu về tập dữ liệu evaluate MathVista

MathVista là một bộ tiêu chuẩn (benchmark) tổng hợp về suy luận Toán học trong bối cảnh Thị giác, được thiết kế để đánh giá khả năng kết hợp giữa hiểu hình ảnh và lập luận toán học của các mô hình AI.

Đặc điểm chính của MathVista:

- Quy mô và Độ đa dạng: MathVista bao gồm 6.141 ví dụ được thu thập từ 31 tập dữ liệu khác nhau.1
- Mục tiêu đánh giá: Đánh giá độ phức tạp đa dạng của thách thức nhận thức thị giác và suy luận toán học.1
- Thành phần dữ liệu mới: Bộ benchmark này giới thiệu ba tập dữ liệu mới, tập trung giải quyết các lĩnh vực thị giác còn thiếu và được tùy chỉnh để đánh giá các dạng suy luận cụ thể:1
    - IQTest: Đánh giá lập luận logic trên các hình vẽ kiểm tra câu đố.
    - FunctionQA: Đánh giá lập luận đại số trên các biểu đồ hàm số.
    - PaperQA: Đánh giá lập luận khoa học với các hình ảnh từ bài báo học thuật.
- Dữ liệu tích hợp: MathVista cũng tích hợp 9 tập dữ liệu MathQA và 19 tập dữ liệu VQA (Visual Question Answering) từ các nghiên cứu trước, nhằm làm phong phú thêm tính đa dạng và độ phức tạp của các thử thách.

Examples of our newly annotated datasets: IQTest, FunctionQA, and PaperQA:



(a) IQTest (b) FunctionQA (c) PaperQA

All the data examples were divided into two subsets: testmini and test.

- testmini: 1,000 examples used for model development, validation, or for those with limited computing resources.
- test: 5,141 examples for standard evaluation. Notably, the answer labels for test will NOT be publicly released.

2. Cấu trúc Prompt và Định dạng Dữ liệu

Dữ liệu được đóng gói theo khuôn dạng *ShareGPT* với các special token của Qwen3-VL, cụ thể:

Finetune với Qwen3-VL-NoThinking:

```
{
```

```
 "messages": [
        {
        "role": "user",
        "content": "[Câu hỏi]\n\n\nOptions: A, B, C, D, E\n<image>"
        },
        {
        "role": "assistant",
        "content": "[Đáp án ngắn — A, B, C, D, E hoặc số]"
        }
 ],
 "images": ["images/xxx.jpg"]
}
```

```
{
 "messages": [
   {
    "role": "user",
     "content": "Which bike is most expensive?\n\n\nOptions: A, B, C, D, E\n<image>"
   },
   {
    "role": "assistant",
    "content": "A"
   }
 ],
```

```
  "images": ["images/2.jpg"]
}
```

Finetune với Qwen3-VL-Thinking:

```
{
  "messages": [
    {
      "role": "user",
      "content": "[Câu hỏi]\n\n\nOptions: A, B, C, D, E\n<image>"
    },
    {
      "role": "assistant",
      "content": "<think>...</think>\n[Đáp án cuối]"
    }
  ],
  "images": ["images/xxx.jpg"]
}
```

```
{
  "messages": [
    {
      "role": "user",
      "content": "Which bike is most expensive?\n\n\nOptions: A, B, C, D,
E\n<image>"
    },
    {
      "role": "assistant",
      "content": "<think>...</think>\nA"
    }
  ],
  "images": ["images/2.jpg"]
}
```

Toàn bộ chuỗi suy luận được bao bởi token <think>...</think>. Phần sau </think> là đáp án ngắn gọn. Thiết kế này tương thích hoàn toàn với

template ReasoningTemplate (qwen3_vl) trong framework LLaMA Factory, đồng thời cho phép suy luận và đáp án được tách biệt rõ ràng trong quá trình đánh giá. Ngoài data thinking, dữ liệu còn lấy thêm phần data không có thinking để hình thành cơ chế hybrid thinking cho model.

3. Kỹ thuật Finetuning

3.1 Mô hình Cơ sở

Mô hình nền được lựa chọn là Qwen3-VL-4B-Instruct và Qwen3-VL-4B-Thinking (Qwen3 Vision-Language, 4 tỷ tham số) –Qwen3-VL một mô hình đa phương thức (multimodal) hỗ trợ xử lý đồng thời văn bản và hình ảnh. Các lý do lựa chọn bao gồm:

- Qwen3-VL được thiết kế đặc biệt cho các bài toán Visual Question Answering (VQA), hỗ trợ sẵn cơ chế *thinking* qua token <think> tích hợp trong tokenizer.
- Kích thước 4B tham số phù hợp với phần cứng huấn luyện thực tế mà vẫn đảm bảo năng lực lập luận đủ mạnh.
- Mô hình có kiến thức nền đa ngôn ngữ (tiếng Anh, tiếng Trung, tiếng Việt), hỗ trợ tốt cho bài toán VQA tiếng Việt.

3.2 Phương pháp Fine-tuning: LoRA

Nghiên cứu sử dụng phương pháp LoRA (Low-Rank Adaptation), một kỹ thuật Parameter-Efficient Fine-Tuning (PEFT) thay vì full fine-tuning toàn bộ tham số.

LoRA chèn các ma trận phân tách hạng thấp $\Delta W = BA$ (trong đó $B \in \mathbb{R}$^{$d \times r$}, $A \in \mathbb{R}$^{$r \times k$}, $r \ll min(d, k)$) vào các lớp tuyến tính của mô hình. Quá trình này chỉ cập nhật một số lượng nhỏ tham số trong quá trình huấn luyện và giữ nguyên toàn bộ trọng số gốc.

Cấu hình LoRA được sử dụng:

Tham số	**Giá trị**
Rank (r)	8
Alpha (α)	16
Scale factor (α/r)	2.0
Target modules	Tất cả các lớp tuyến tính (all)
Finetuning type	Lora
Gradient checkpointing	Bật

Việc áp dụng LoRA lên tất cả các module tuyến tính (bao gồm Q, K, V projection, attention output, các lớp MLP) giúp mô hình thích nghi toàn diện hơn so với việc chỉ nhắm vào một số module nhất định. Gradient checkpointing được bật để giảm yêu cầu VRAM.

3.3 . Hàm Mất mát

Hàm mất mát sử dụng là **Causal Language Modeling Loss** (Cross-Entropy Loss trên chuỗi output theo hướng nhân quả), tuân theo phương pháp SFT tiêu chuẩn:

$$\mathcal{L} = -\sum_{t=1}^{T} \log P(y_t \mid y_{<t}, x)$$

Trong đó:

- x là input (hình ảnh + câu hỏi).
- y_t là token thứ t trong response.

Loss được tính trên toàn bộ phần response của model — bao gồm cả chuỗi suy luận trong <think>...</think> và đáp án cuối — trong khi phần prompt người dùng được mask để không đóng góp vào gradient. Điều này buộc mô hình học đồng thời cả quá trình suy luận trung gian lẫn đáp án cuối cùng.

4. Thiết lập tham số và môi trường

4.1 Siêu tham số

Tham số	**Giá trị**
Learning rate	$5\ x\ 10^{-5}$
LR Scheduler	Cosine Annealing
Warmup Ratio	0.1 (10% bước đầu)
Batch size / GPU	1
Gradient accumulation steps	16
Effective batch size	16
Số epoch	3
Optimizer	AdamW
Max gradient norm	1.0
Precision	BFloat16 (bf16)
Max sequence length	8,192 tokens
Validation split	335 mẫu
Đánh giá (Eval steps)	Mỗi 100 bước

Image max pixels	262.144 (~512×512)

4.2 Môi trường Huấn luyện

Phần cứng:

Thành phần	**Cấu hình**
GPU	NVIDIA RTX A6000
Mixed Precision	BFloat16
Gradient Checkpointing	Bật

Phần mềm:

Thư viện	**Phiên bản / Vai trò**
LLaMa Factory	Framework finetuning
Hugging Face Transformers	Load/save model và tokenizer
PEFT	Triển khai LoRA adapter
TRL	SFT Trainer
Gemini 3.1 Flash lite (OpenRouter)	Sinh dữ liệu thinking chain
Python	≥ 3.9

V. Kết quả và đánh giá

5. Phương pháp và độ đo đánh giá

Phần thực nghiệm này tập trung đánh giá hiệu quả của hai chiến lược fine-tuning trên bộ dữ liệu **MathVision** với backbone **Qwen3-VL-4B-Instruct và Qwen3-VL-4B-Thinking**:

- **Qwen3-VL-4B-Instruct**: huấn luyện trên dữ liệu chỉ chứa đáp án cuối.
- **Qwen3-VL-4B-Thinking**: huấn luyện trên dữ liệu có chuỗi suy luận dạng <think> ... </think> kèm đáp án.

Đánh giá được thực hiện trên tập mathvision testmini gồm 304 mẫu thuộc tập MathVison train để đánh giá xem lora apdapter có học được format và cấu trúc của dữ liệu train không. Sau đó mô hình được đánh giá trên tập MathVista testmini để đánh giá độ chính xác của mô hình sau khi finetune. Chỉ số chính sử dụng để so sánh là exact-match accuracy giữa đáp án dự đoán và đáp án chuẩn sau bước chuẩn hóa đầu ra. Bốn cấu hình được so sánh gồm:

- **Qwen3-VL-4B-Instruct**: mô hình gốc, không fine-tuning, không bật thinking.
- **Qwen3-VL-4B-Thinking**: mô hình gốc, không fine-tuning, nhưng chạy với reasoning template.
- **Qwen3-VL-4B-Instruct finetuned**: mô hình fine-tuning trên dữ liệu answer-only.

- **Qwen3-VL-4B-Thinking finetuned**: mô hình fine-tuning trên dữ liệu có chain-of-thought.

Lưu ý quan trọng: trong quá trình phân tích, script eval_mathvision.py đã được cải tiến để bóc tách đáp án tốt hơn cho đầu ra dạng reasoning dài. Vì vậy, các kết quả accuracy được báo cáo dưới đây phản ánh đúng hơn khả năng trả lời cuối cùng của mô hình, đặc biệt với nhánh thinking.

Xác nhận lại vai trò của hai tập dữ liệu đánh giá:

MathVision (304 mẫu): Là tập nhỏ được trích từ dữ liệu huấn luyện, dùng để kiểm tra mô hình có học đúng format fine-tuning và fit với dữ liệu train hay không.

MathVista (1000 mẫu): Là tập test độc lập, dùng để đánh giá độ chính xác thực tế và khả năng tổng quát hóa của mô hình.

6. Kết quả định lượng

2.1 Đánh giá trên tập MathVision - testmini

Metric	Qwen3-VL Instruct	Qwen3-VL Instruct FT	Qwen3-VL Thinking	Qwen3-VL Thinking FT
Exact Match	80 / 304	163 / 304	60 / 304	102 / 304
Accuracy	29.61%	53.95%	19.74%	33.55%
BLEU-4	0.1832	91.5421	0.3178	0.2687
ROUGE-1	3.3234	91.3544	5.5454	44.3863
ROUGE-2	0.1203	92.2645	0.9824	0.2421
ROUGE-L	1.0745	94.6515	1.3251	11.7047

Runtime (s)	10537.34	114.52	12395.09	6819.89
Samples/s	0.032	2.925	0.027	0.049

Đánh giá hiệu quả khớp dữ liệu (Internal Validation) trên MathVision

Kết quả thực nghiệm định lượng trên tập dữ liệu huấn luyện (304 mẫu) nhằm kiểm chứng khả năng đáp ứng format và độ hội tụ của mô hình cho thấy:

Hiệu quả vượt trội của chiến lược Fine-tuned-base

- Qwen3-VL Instruct FT (FT-Base) đạt hiệu quả tốt nhất với 53,95% độ chính xác.
- Nhận xét: Sự bùng nổ của các chỉ số ngôn ngữ như BLEU-4 (91.54) và ROUGE-L (94.65) minh chứng rằng mô hình đã "fit" gần như hoàn hảo với dữ liệu. Do tập dữ liệu này chỉ bao gồm câu hỏi và đáp án ngắn gọn, mô hình dễ dàng học thuộc lòng và tuân thủ tuyệt đối format mục tiêu, dẫn đến bước nhảy vọt về cả độ chính xác lẫn tốc độ xử lý (Runtime chỉ 114.52s).

Khả năng "bắt chước" của chiến lược Fine-tuned-thinking

- Qwen3-VL Thinking FT (FT-Thinking) đạt 33,55%. Mặc dù thấp hơn bản Instruct 20,4%, nhưng các chỉ số nội hàm cho thấy tiềm năng rất lớn.
- Nhận xét: Với đặc thù dữ liệu "thinking" rất dài và phức tạp, việc mô hình ghi nhớ chính xác từng ký tự đáp án cuối (Exact Match) khó khăn hơn nhiều so với bản Base. Tuy nhiên, chỉ số ROUGE-1 tăng vọt

lên 44.38 (so với 5.54 của bản gốc) chứng minh mô hình đã bắt đầu "ăn" theo format suy luận và bắt chước rất giống luồng tư duy mà bạn đã sắp đặt, dù chưa đạt tới độ hội tụ tuyệt đối về mặt con số như bản đáp án ngắn.

So sánh mức độ tăng trưởng (Mốc nền: Qwen3-VL Thinking 19,74%)

- Qwen3-VL Instruct (Base): Tăng 9,87%.
- Qwen3-VL Thinking FT: Tăng 13,81%.
- Qwen3-VL Instruct FT: Tăng mạnh nhất với 34,21%.

Thử nghiệm nội bộ xác nhận mô hình đã fit được data và đi đúng hướng về mặt định dạng (format alignment):

- Chiến lược FT-Base: Thể hiện khả năng tối ưu hóa điểm số cực mạnh nhờ cấu trúc dữ liệu đơn giản, tinh gọn, phù hợp cho các tác vụ cần câu trả lời nhanh và chính xác theo mẫu.
- Chiến lược FT-Thinking: Dù độ phức tạp của dữ liệu dài gây rào cản cho việc khớp đáp án (Exact Match), mô hình đã thể hiện khả năng mô phỏng lại cấu trúc tư duy một cách rõ rệt. Việc Runtime của bản FT-Thinking (6819.89s) thấp hơn nhiều so với bản Thinking gốc (12395.09s) cho thấy mô hình sau khi tune đã học được cách "tư duy" gãy gọn và bám sát mục tiêu hơn.

Giả định: Tuy bản Base đang dẫn đầu về con số trên tập train, nhưng bản Thinking với khả năng "mô phỏng tư duy" này được kỳ vọng sẽ bộc lộ ưu

thế về tính logic và khả năng giải quyết các bài toán lạ khi đánh giá trên tập MathVista, nơi mà việc học thuộc lòng không còn tác dụng.

2.2 Đánh giá trên tập MathVision - testmini

Cấu hình	Exact match	Accuracy
Qwen3-Vl-Thinking	209 / 1000	20,90%
Qwen3-Vl-Thinking finetuned	**261/ 1000**	**26,10%**
Qwen3-Vl-Intruct	253/ 1000	25,30%
Qwen3-Vl-Intruct finetuned	168/ 1000	16,80%

Kết quả đánh giá trên 1000 mẫu (testmini) cho thấy sự thay đổi đáng kể về hiệu suất giữa các cấu hình, đặc biệt là khi so sánh với kết quả đánh giá trên tập nhỏ (304 mẫu) được đề cập trong tài liệu.1. Phân tích Hiệu suất Tổng thể

Cấu hình	Accuracy	Xếp hạng (1000 mẫu)	Xếp hạng (335 mẫu)
Qwen3-Vl-Thinking finetuned	**26,10%**	**Hạng 1**	Hạng 2 (30,45%)
Qwen3-Vl-Intruct (Base)	25,30%	Hạng 2	Hạng 3 (20,88%)
Qwen3-Vl-Thinking (Base)	20,90%	Hạng 3	Hạng 4 (17,91%)
Qwen3-Vl-Intruct finetuned	**16,80%**	**Hạng 4**	**Hạng 1 (48,96%)**

Cấu hình tốt nhất: Mô hình Qwen3-Vl-Thinking finetuned đạt độ chính xác cao nhất (26,10%) trên tập 1000 mẫu này. Điều này cho thấy chiến lược

fine-tuning sử dụng chuỗi suy luận (<think>...</think>) đã cải thiện năng lực suy luận của mô hình cơ sở (tăng từ 20,90% lên 26,10%).

Sự suy giảm nghiêm trọng của Fine-tuned Instruct: Cấu hình Qwen3-Vl-Intruct finetuned bất ngờ cho kết quả thấp nhất (16,80%). Mức giảm này đặc biệt đáng chú ý vì cấu hình tương đương này là cấu hình tốt nhất trên tập 335 mẫu (đạt 48,96%).

Sự thay đổi thứ hạng đột ngột của cấu hình fine-tuned cho thấy:

Hiệu quả của chiến lược Thinking: Trên tập dữ liệu lớn hơn (1000 mẫu), việc huấn luyện mô hình sinh chuỗi suy luận (Thinking) đã chứng minh tính ổn định và hiệu quả hơn (tăng 5,2% so với Base).

Vấn đề Overfitting hoặc Tính ổn định: Hiệu suất của Qwen3-Vl-Intruct finetuned bị giảm mạnh từ 48,96% xuống 16,80% khi chuyển từ 335 mẫu sang 1000 mẫu. Điều này ủng hộ giả định rằng mô hình này có thể đã overfitting nghiêm trọng với tập huấn luyện trả lời ngắn, hoặc tập 1000 mẫu có độ phức tạp cao hơn, khiến chiến lược huấn luyện chỉ sinh đáp án ngắn không còn hiệu quả.

Cả hai mô hình cơ sở (Qwen3-Vl-Intruct và Qwen3-Vl-Thinking) đều đạt hiệu suất tương đối thấp (25,30% và 20,90%), cho thấy nhu cầu fine-tuning là cần thiết để xử lý các bài toán toán học trong bối cảnh thị giác.

Mô hình Base Instruct có hiệu suất tốt hơn Base Thinking (25,30% so với 20,90%).

Dựa trên kết quả đánh giá lại trên 1000 mẫu:

Chiến lược Fine-tuned Thinking (huấn luyện sinh chuỗi suy luận) là phương pháp tối ưu nhất cho tập MathVision/MathVista - testmini (1000 mẫu), vượt

trội so với chiến lược chỉ huấn luyện sinh đáp án ngắn (Instruct) trên cùng tập dữ liệu này.

3. Đánh giá định tính

Bên cạnh các chỉ số định lượng, việc quan sát trực tiếp kết quả sinh ra (output) của mô hình cung cấp cái nhìn sâu sắc về cách thức hoạt động và khả năng tư duy của từng cấu hình:

Qwen3-VL Instruct FT (FT-Base): "Cỗ máy học thuộc"

Đặc điểm sinh văn bản: Trực diện, ngắn gọn và bám sát tuyệt đối vào định dạng câu trả lời cuối (chỉ đưa ra đáp án A, B, C, D hoặc một con số duy nhất).

Ưu điểm: Không có từ ngữ thừa, loại bỏ hoàn toàn hiện tượng "ảo giác" (hallucination) trong quá trình diễn giải.

Nhược điểm: Hoàn toàn là một "hộp đen" (black box). Khi mô hình đưa ra đáp án sai, chúng ta không thể truy vết được lỗi nằm ở khâu trích xuất hình ảnh (nhìn sai) hay khâu tính toán (tư duy sai). Khả năng cao mô hình đang dùng trí nhớ trọng số (weight memory) để map thẳng từ hình ảnh/câu hỏi ra đáp án thay vì thực sự giải quyết bài toán.

Qwen3-VL Thinking FT (FT-Thinking): "Học sinh đang tập suy luận"

Đặc điểm sinh văn bản: Output dài, có chia bước rõ ràng (ví dụ: phân tích giả thiết, thiết lập công thức, tính toán và kết luận). Mô hình đã bắt chước thành công văn phong và luồng logic (Chain-of-Thought) từ dữ liệu được sinh ra bởi Gemini.

Ưu điểm: Khả năng diễn giải (Interpretability) cao. Ngay cả khi Exact Match báo sai, nhiều trường hợp mô hình vẫn phân tích đúng bản chất hình học

hoặc đồ thị, chỉ sai ở bước tính toán số học cuối cùng. Việc mô hình tự động dừng suy luận (biểu hiện qua thời gian Runtime giảm một nửa so với bản gốc) cho thấy nó đã học được cách kiểm soát độ dài của chuỗi tư duy, không còn bị lan man vô định.

Nhược điểm: Đôi khi cấu trúc câu còn rập khuôn, và do phải xử lý chuỗi token dài, mô hình dễ bị mất tập trung (attention decay) ở các bước cuối, dẫn đến kết luận sai lệch so với logic ban đầu.

---

4. Phân tích lỗi

Việc phân tích các mẫu dự đoán sai (chiếm $\sim 46\%$ ở bản FT-Base và $\sim 66\%$ ở bản FT-Thinking) giúp xác định các điểm nghẽn hiện tại của mô hình trên tập MathVision:

Lỗi định dạng đáp án (Formatting / Extraction Error):

Biểu hiện: Đặc biệt phổ biến ở bản FT-Thinking. Mô hình suy luận đúng và đưa ra kết quả cuối cùng đúng về mặt giá trị, nhưng bị hệ thống chấm là sai do không khớp chuỗi ký tự (Exact Match).

Ví dụ: Ground truth là 5, nhưng mô hình sinh ra x = 5, Đáp án là 5 hoặc định dạng phân số $1/2$ thay vì 0.5. Điều này giải thích tại sao Accuracy thấp nhưng ROUGE-1 lại khá cao.

Lỗi nhận thức thị giác (Visual Perception Error):

Biểu hiện: Cả hai mô hình đều gặp khó khăn trong việc trích xuất chính xác thông tin từ các biểu đồ phức tạp (như biểu đồ phân tán có quá nhiều điểm), hoặc đọc sai các ký tự/con số nhỏ (OCR fail) nằm lẫn trong các hình học không gian.

Hậu quả: Sai số liệu ngay từ bước đầu tiên (premise), dẫn đến toàn bộ chuỗi tính toán phía sau của bản FT-Thinking bị sụp đổ, dù công thức áp dụng là đúng.

Lỗi đứt gãy logic và tính toán (Reasoning & Calculation Error):

Biểu hiện (chủ yếu ở FT-Thinking): Mô hình thiết lập phương trình đúng từ hình ảnh, nhưng lại giải sai phương trình bậc nhất/bậc hai cơ bản. Hoặc mô hình đang suy luận theo hướng A, nhưng đột ngột rẽ sang hướng B ở giữa văn bản mà không có sự liên kết.

Nhiễu từ dữ liệu huấn luyện (Data Noise):

Biểu hiện: Vì dữ liệu "thinking" được tạo ra tự động từ Gemini 1.5 Flash (hoặc Lite), chắc chắn sẽ tồn tại một tỷ lệ bài giải có luồng suy luận cồng kềnh, đi đường vòng hoặc thậm chí là sai logic ngầm. Việc ép Qwen3-VL học theo các chuỗi bị nhiễu này khiến mô hình đôi khi tạo ra các bước giải thừa thãi, làm tăng nguy cơ sai số cuối cùng.